

课程概述

Nodejs 高级编程

大前端开发的必备技能

Nodejs 可以做什么？

- 轻量级、高性能的 Web 服务
- 前后端 JavaScript 同构开发
- 便捷高效的前端工程化

Nodejs的架构和运行过程

Nodejs 异步IO和事件驱动

Nodejs 单线程

Nodejs 实现 API 服务

Nodejs 核心模块及 API 使用

Nodejs 架构

Natives modules

Node C/C++ Bindings

V8

libuv

c-ares
(DNS)

http
parser

zlib
(compression)

.....

CPU/RAM/DISK OS

Natives modules

- 当前层内容由 JS 实现
- 提供应用程序可直接调用库，例如 fs 、 path、 http 等
- JS 语言无法直接操作底层硬件设置

Builtin modules “胶水层”

V8

libuv

c-ares
(DNS)

http
parser

zlib
(compression)

.....

底层

- V8：执行 JS 代码，提供桥梁接口
- Libuv：事件循环、事件队列、异步IO
- 第三方模块：zlib 、http、c-ares等

JavaScript @ Node.js

ECMAScript

Node APIs

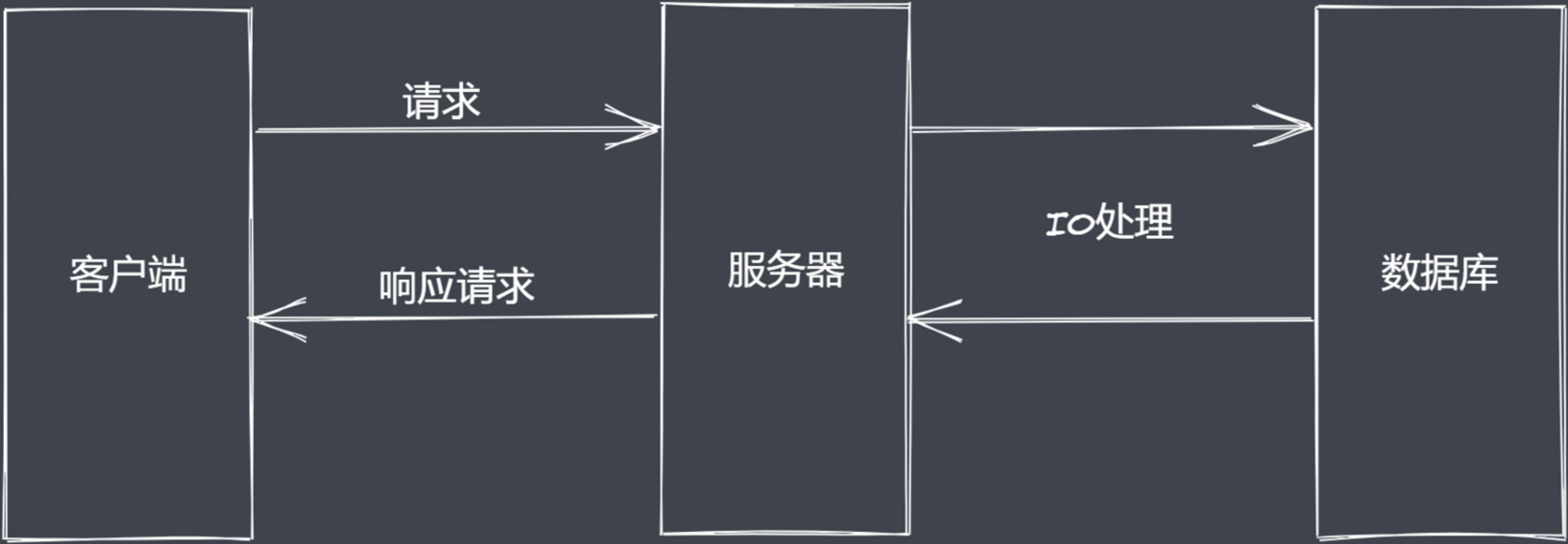
fs

net

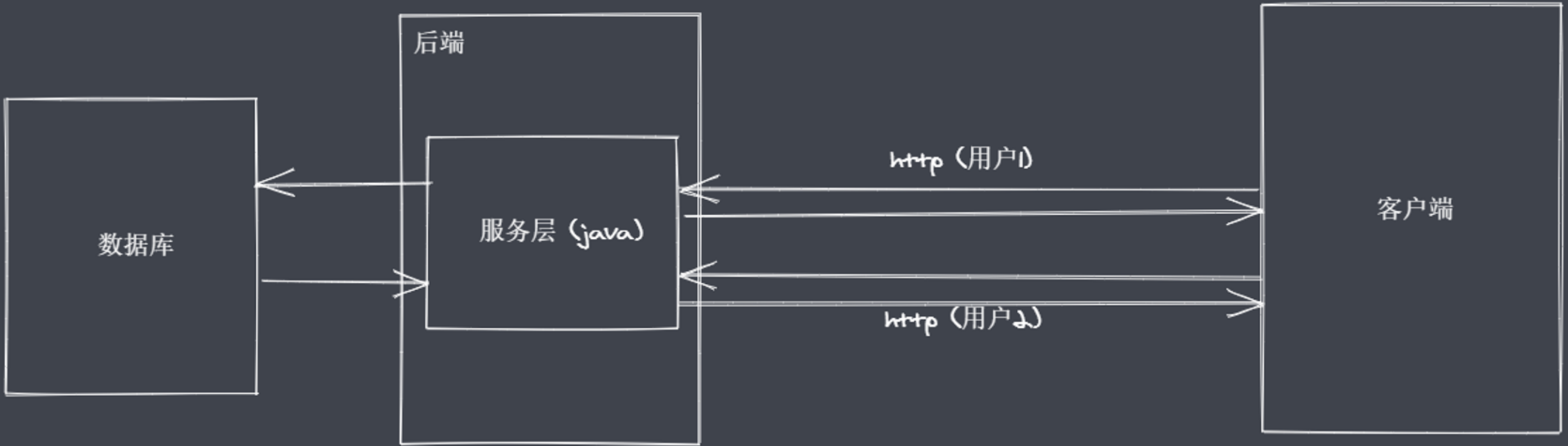
process

为什么是 Nodejs

Nodejs 慢慢演化为一门服务端“语言”



IO 是计算机操作过程中最缓慢的环节



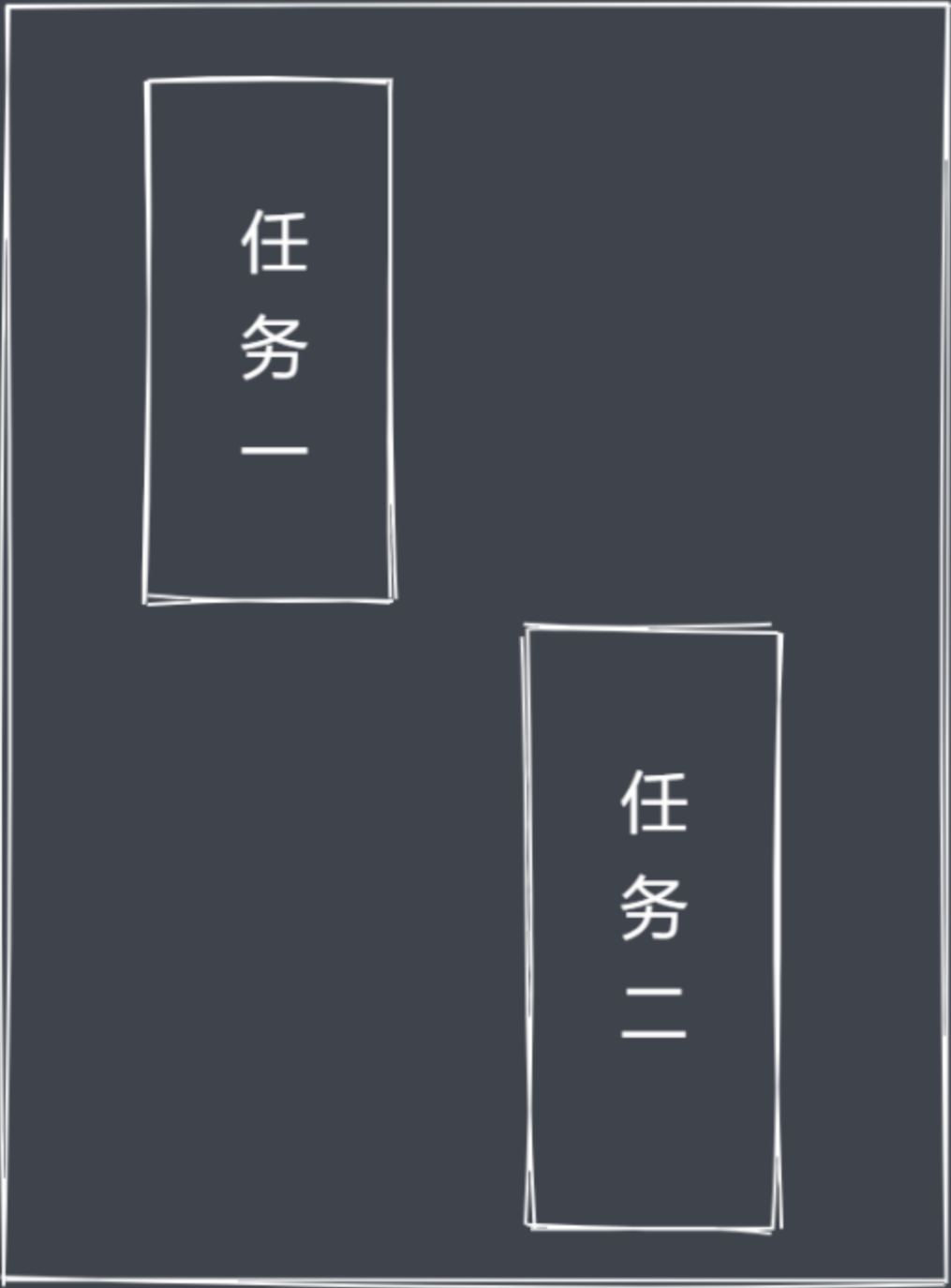
Reactor 模式，单线程完成多线程工作

Reactor 模式下实现异步IO、事件驱动

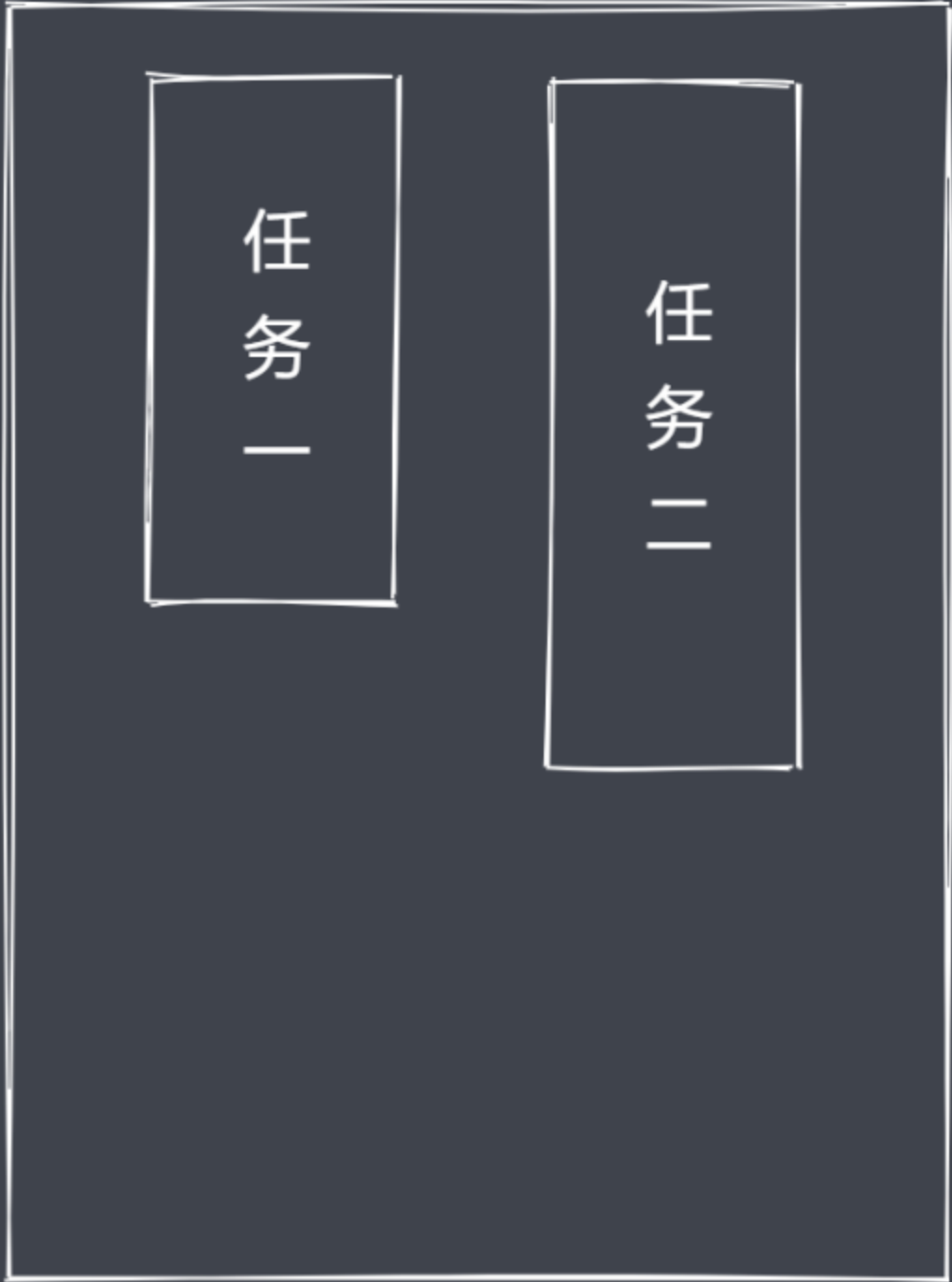
Nodejs 更适用于 IO 密集型高并发请求

Nodejs 异步 IO

同步



异步



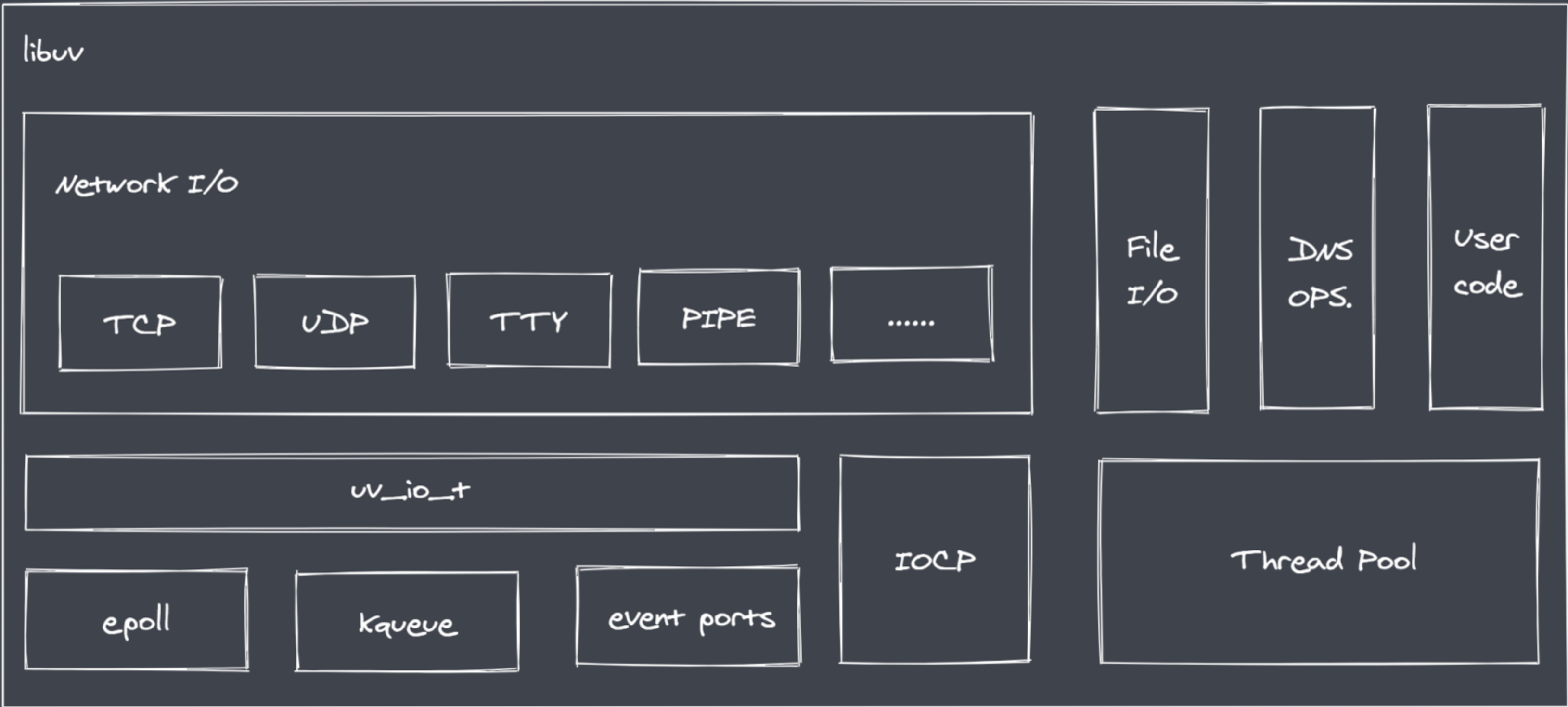
非阻塞 IO

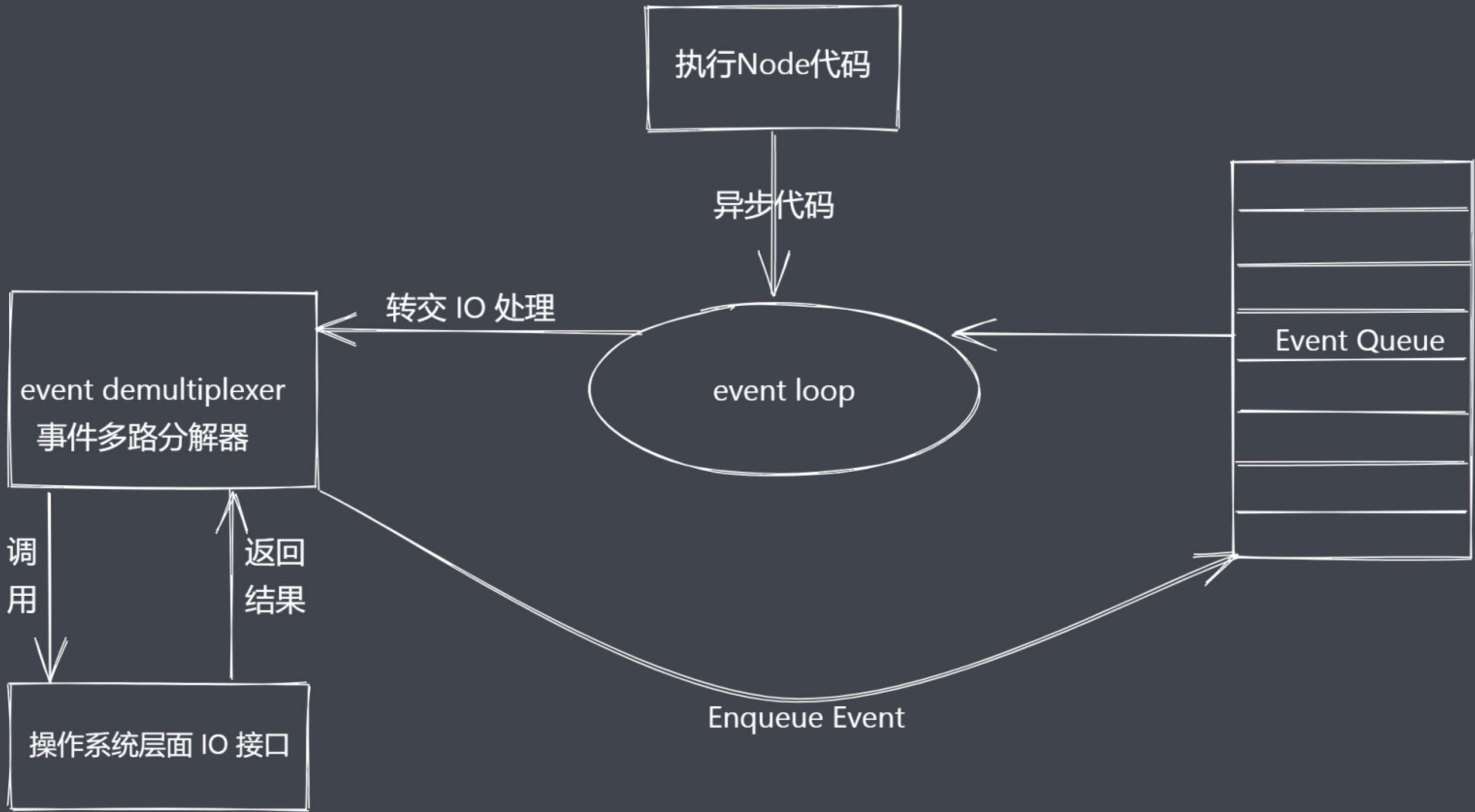
阻塞 10

重复调用 10 操作，判断是 10 是否结
束

read、select、poll、kqueue、event ports

期望实现无须主动判断的非阻塞 IO





异步 IO 总结

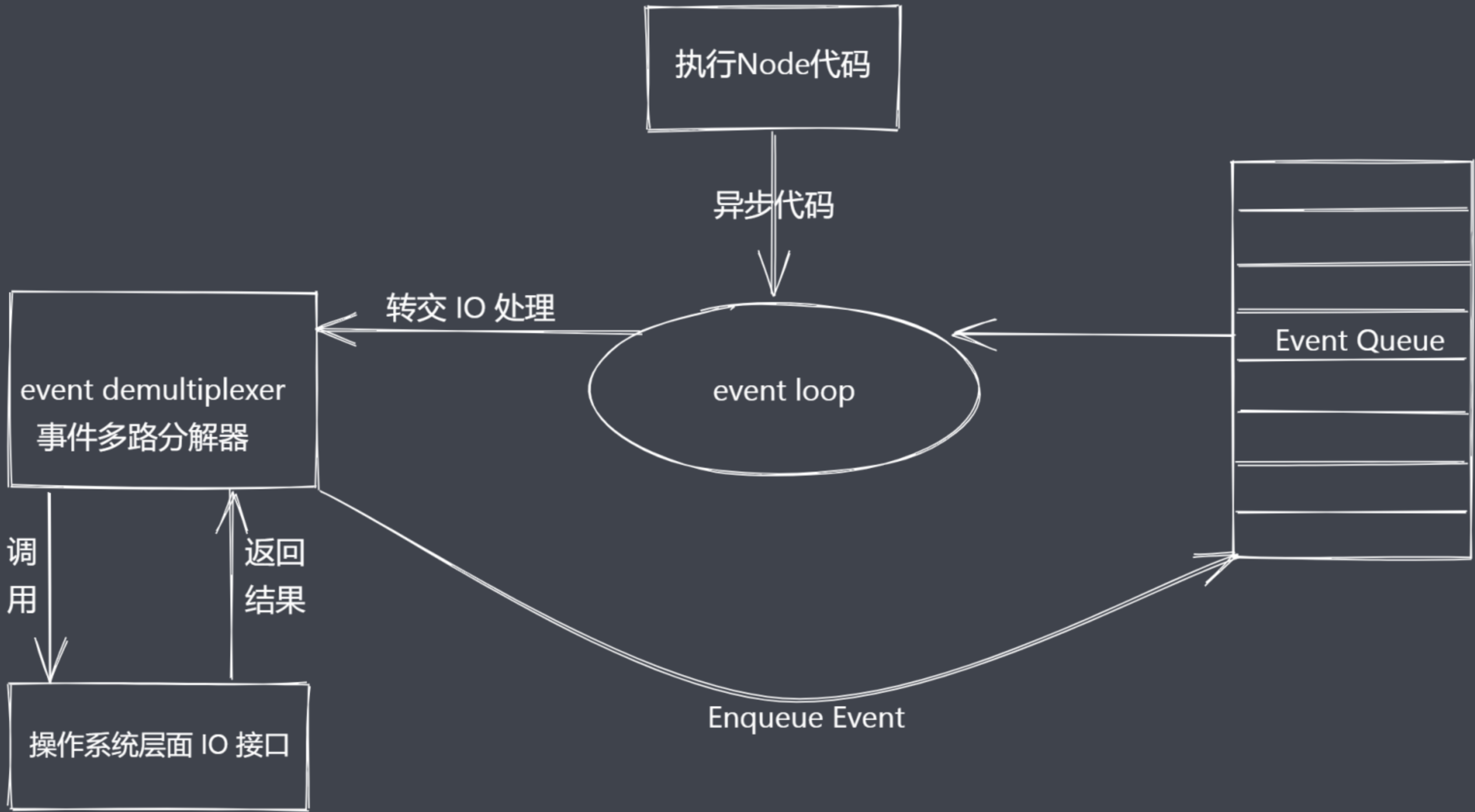
- IO 是应用程序的瓶颈所在
- 异步 IO 提高性能无采原地等待结果返回
- IO 操作属于操作系统级别，平台都有对应实现
- Nodejs 单线程配合事件驱动架构及libuv 实现了异步 IO

Nodejs 事件驱动架构

事件驱动架构是软件开发中的通用模式

事件驱动、发布订阅、观察者

主体发布消息，其它实例接收消息



Nodejs 单线程

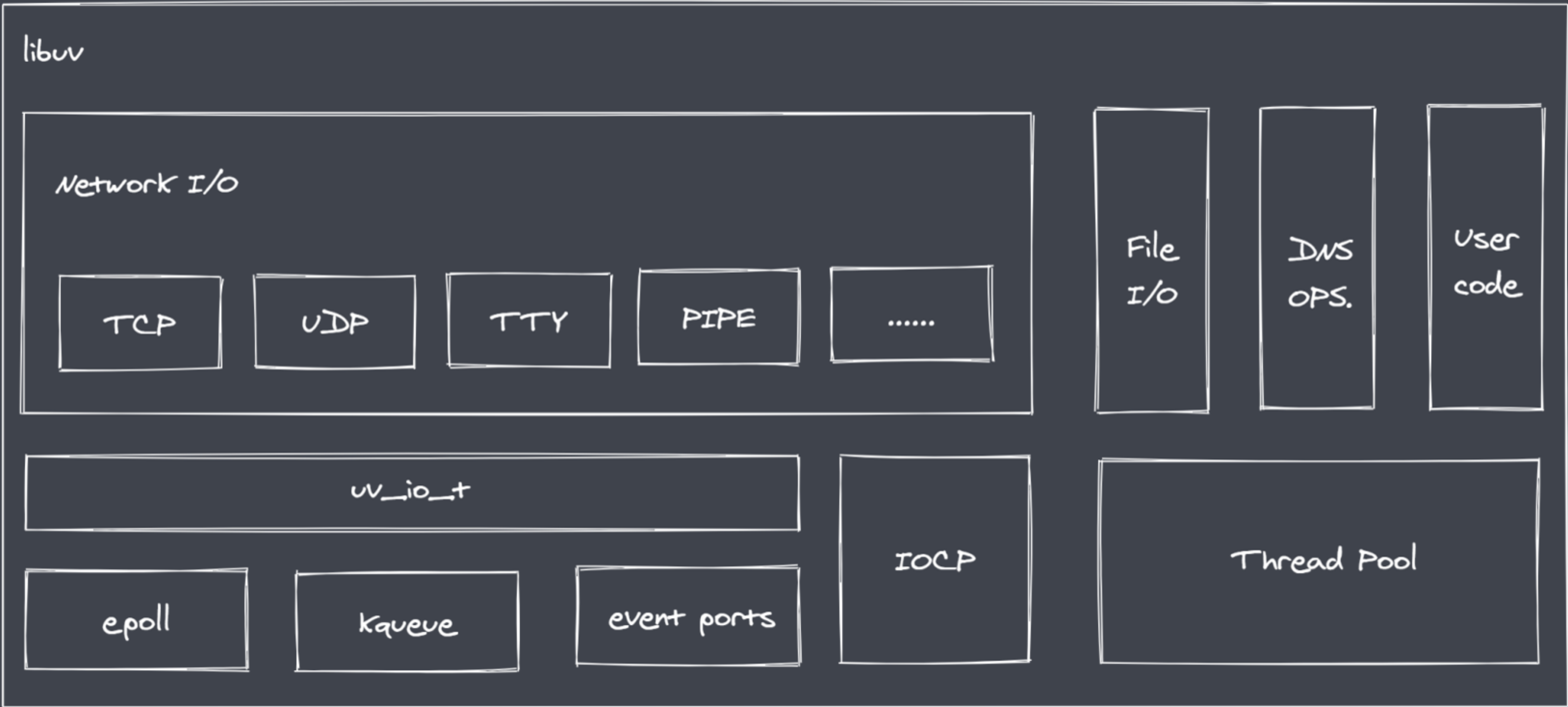
异步IO、事件驱动、事件循环

使用 JS 实现高效可伸缩的高性能 Web 服务

单线程如何实现高并发？

异步非阻塞 IO 配合事件回调通知（切PPT）

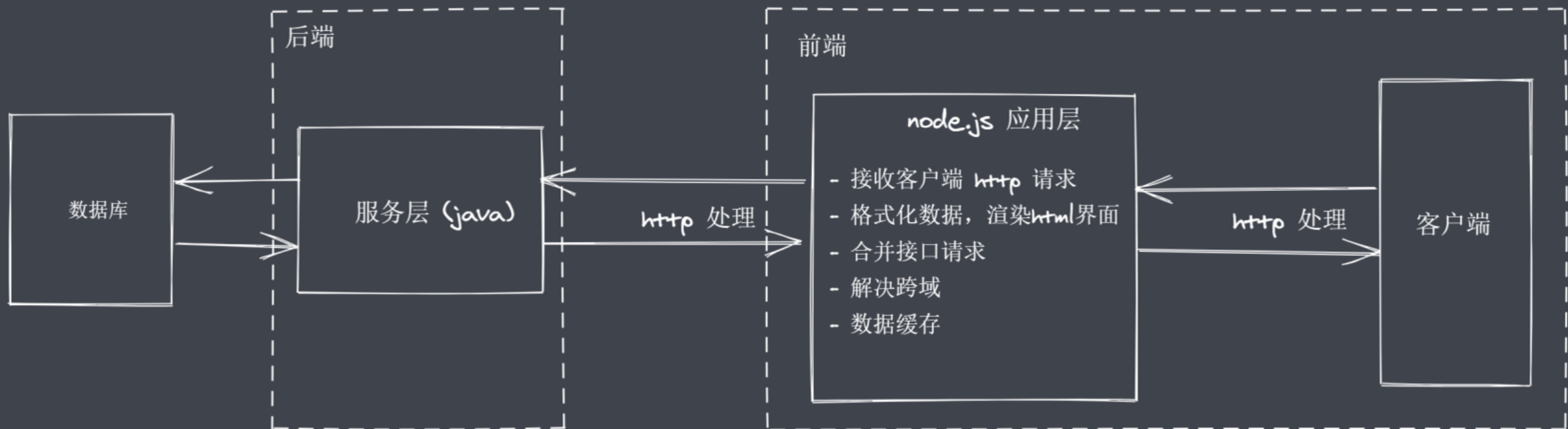
Nodejs 主线程是单线程



Nodejs 应用场景

10 密集型高并发请求

Node.js 做为中间层



操作数据库提供 API 服务

实时聊天应用程序

Nodejs 更适合 IO 密集型任务

Nodejs 实现 API 服务

全局对象

从哪入手，如何开始？

Nodejs 全局对象

- 与浏览器平台的 window 不完全相同
- Nodejs 全局对象上挂载许多属性

全局对象是 JavaScript 中的特殊对象

Nodejs 中全局对象是 `global`

Global 的根本作用就是作为宿主

全局对象可以看做是全局变量的宿主

Nodejs 常见全局变量

- `__filename`: 返回正在执行脚本文件的绝对路径
- `__dirname`: 返回正在执行脚本所在目录
- `timer` 类函数: 执行顺序与事件循环间的关系
- `process`: 提供与当前进程互动的接口
- `require`: 实现模块的加载
- `module`、`exports`: 处理模块的导出

全局变量之 process

无须 require 可直接使用

获取进程信息

执行进程操作

内置模块之 path

内置模块，require 之后直接使用

用于处理文件/目录的路径

掌握 path 模块中常见的 API

path 模块常用 API

- `basename()` 获取路径中基础名称
- `dirname()` 获取路径中目录名称
- `extname()` 获取路径中扩展名称
- `isAbsolute()` 获取路径是否为绝对路径
- `join()` 拼接多个路径片段

path 模块常用 API

- `resolve()` 返回绝对路径
- `parse()` 解析路径
- `format()` 序列化路径
- `normalize()` 规范化路径

全局变量之 Buffer

Buffer 缓冲区

Buffer 让 JavaScript 可以操作二进制

Buffer 是什么？在哪？做什么？

二进制数据、流操作、Buffer

JavaScript 语言起初服务于浏览器平台

Nodejs 平台下 JavaScript 可实现 10

10 行为操作的就是二进制数据

Stream 流操作并非 Nodejs 独创

流操作配合管道实现数据分段传输

数据的端到端传输会有生产者和消费者

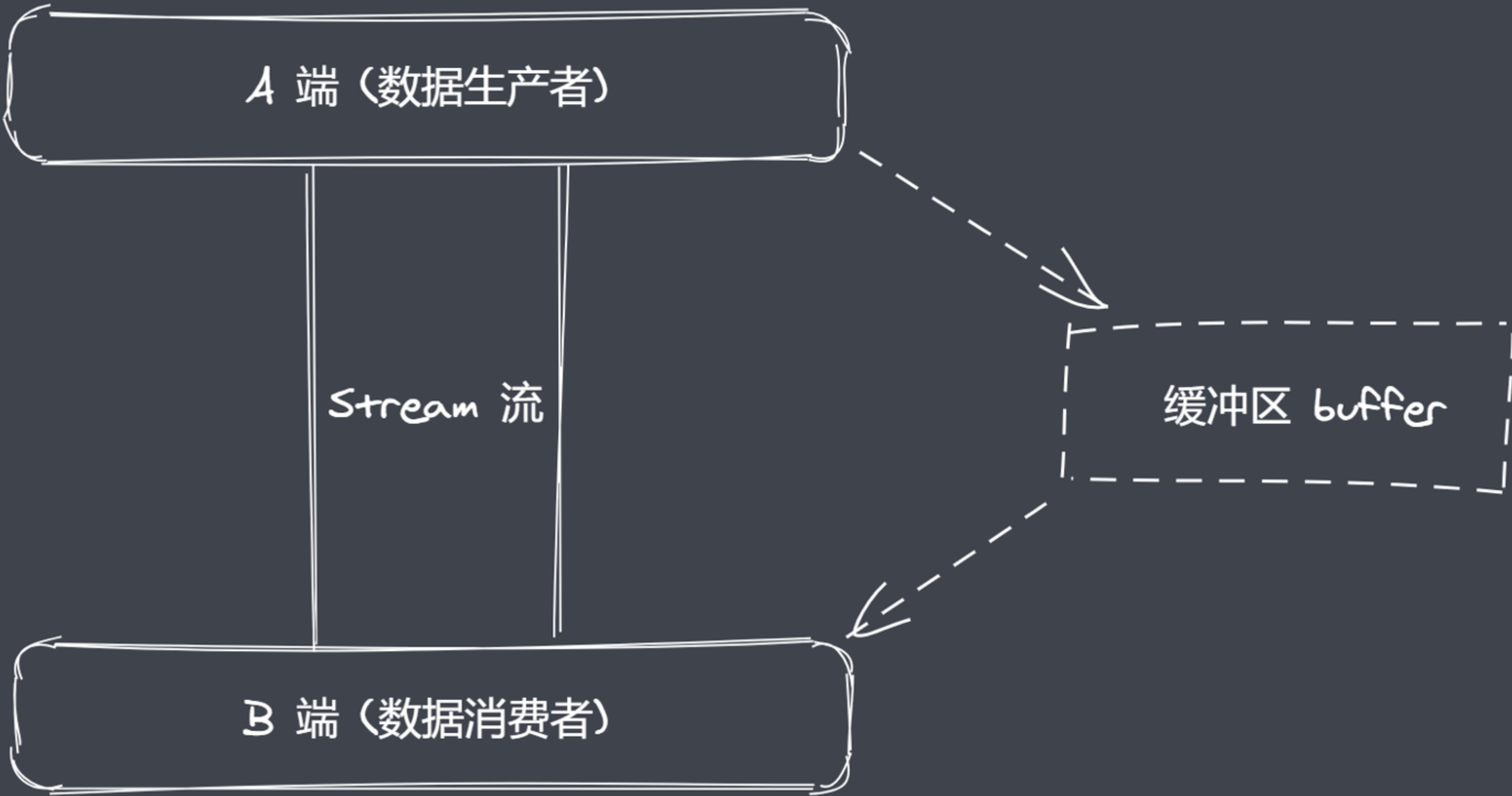
生产和消费的过程往往存在等待

产生等待时数据存放在哪？

Nodejs 中 Buffer 是一片内存空间

Buffer 总结

- 无须 require 的一个全局变量
- 实现 Nodejs 平台下的二进制数据操作
- 不占据 V8 堆内存大小的内存空间
- 内存的使用由 Node来控制，由 V8 的 GC 回收
- 一般配合 Stream 流使用，充当数据缓冲区



创建 Buffer

Buffer 是 Nodejs 的内置类

了解如何创建 Buffer 实例

创建 Buffer 实例

- `alloc`: 创建指定字节大小的 `buffer`
- `allocUnsafe`: 创建指定大小的 `buffer` (不安全)
- `from` : 接收数据, 创建 `buffer`

Buffer 实例方法

Buffer 实例方法

- `fill`: 使用数据填充 `buffer`
- `write`: 向 `buffer` 中写入数据
- `toString`: 从 `buffer` 中提取数据
- `slice`: 截取 `buffer`
- `indexOf`: 在 `buffer` 中查找数据
- `copy`: 拷贝 `buffer` 中的数据

Buffer 静态方法

Buffer 静态方法

- `concat`: 将多个 `buffer` 拼接成一个新的 `buffer`
- `isBuffer`: 判断当前数据是否为 `buffer`

自定义 Buffer 之 split

FS 模块

一个是缓冲区，一个是数据流

Buffer 、Stream 与 FS 有什么关系？

FS 又是什么？

FS 是内置核心模块，提供文件系统操作的
API



FS 模块结构

- FS 基本操作类
- FS 常用 API

权限位、标识符、文件描述符

用户对于文件所具备的操作权限

权限位	文件所有者			文件所属组			其它用户		
权限项	读	写	执行	读	写	执行	读	写	执行
字符表示	r	w	x	r	w	x	r	w	x
数字表示	4	2	1	4	2	1	4	2	1


```
λ ls -al
total 21
drwxr-xr-x 1 yairu 197609 0 11月 6 17:38 ./
drwxr-xr-x 1 yairu 197609 0 11月 3 16:53 ../
-rw-r--r-- 1 yairu 197609 249 11月 2 14:17 01-event-driven.js
-rw-r--r-- 1 yairu 197609 320 11月 2 15:03 02-single-thread.js
-rw-r--r-- 1 yairu 197609 324 11月 3 19:13 04-node-global.js
-rw-r--r-- 1 yairu 197609 698 11月 5 15:45 05-process.js
-rw-r--r-- 1 yairu 197609 2698 11月 6 14:57 06-path.js
-rw-r--r-- 1 yairu 197609 539 11月 6 19:10 07-buffer.js
-rw-r--r-- 1 yairu 197609 12 11月 5 16:58 test.txt
```

Nodejs 中 flag 表示对文件操作方式

常见 flag 操作符

- r: 表示可读

常见 flag 操作符

- r: 表示可读
- w: 表示可写

常见 flag 操作符

- r: 表示可读
- w: 表示可写
- s: 表示同步

常见 flag 操作符

- r: 表示可读
- w: 表示可写
- s: 表示同步
- +: 表示执行相反操作

常见 flag 操作符

- r: 表示可读
- w: 表示可写
- s: 表示同步
- +: 表示执行相反操作
- x: 表示排它操作

常见 flag 操作符

- r: 表示可读
- w: 表示可写
- s: 表示同步
- +: 表示执行相反操作
- x: 表示排它操作
- a: 表示追加操作

fd 就是操作系统分配给被打开文件的标识

fs 介绍总结

- fs 是 Nodejs 中内置核心模块
- 代码层面上 fs 分为基本操作类和常用 API
- 权限位、标识符、操作符

文件读写与拷贝操作

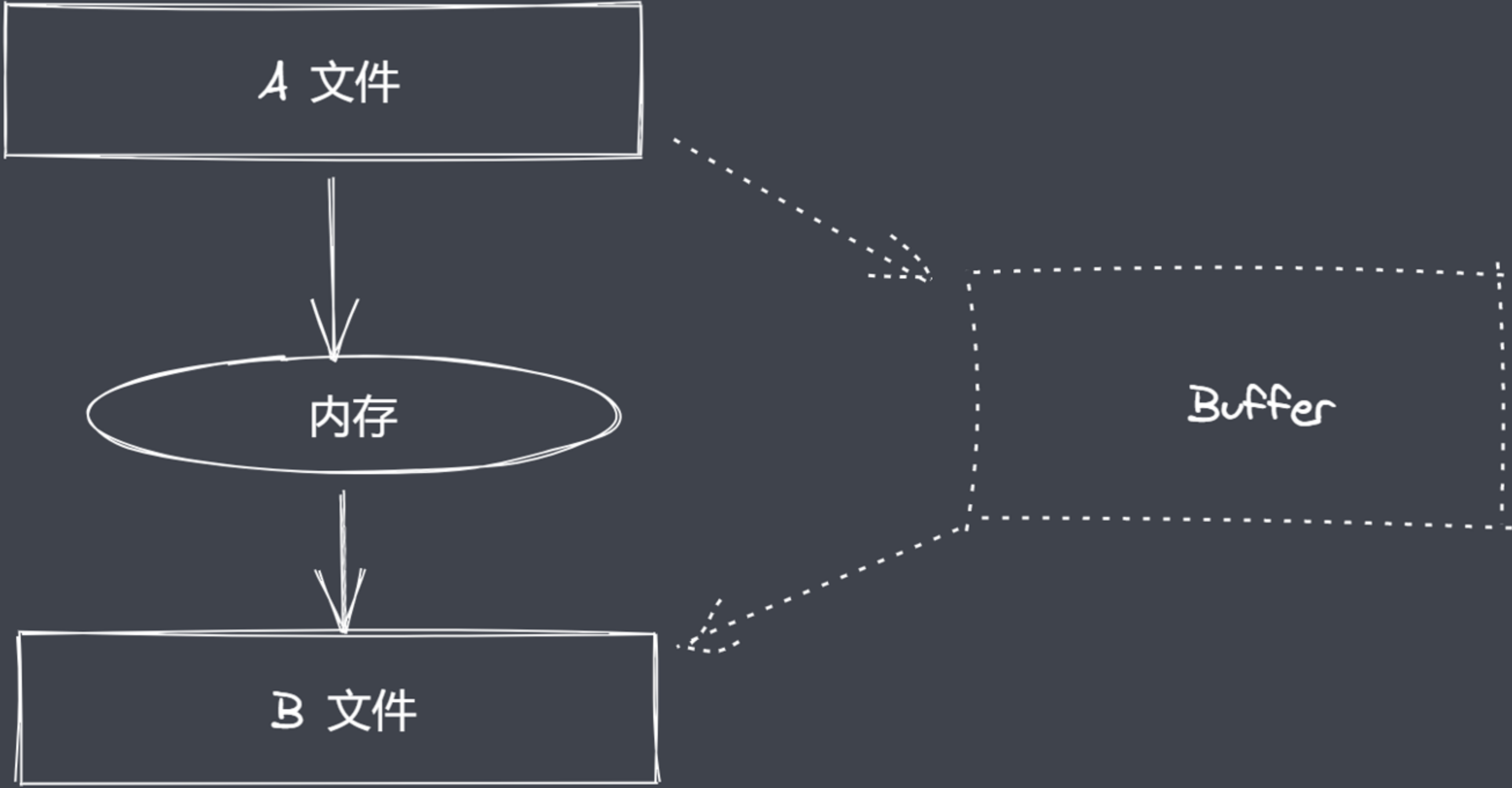
文件操作 API

- `readFile`: 从指定文件中读取数据
- `writeFile`: 向指定文件中写入数据
- `appendFile`: 追加的方式向指定文件中写入数据
- `copyFile`: 将某个文件中的数据拷贝至另一文件
- `watchFile`: 对指定文件进行监控

文件操作实现 md 转 html

文件打开与关闭

大文件读写操作



文件拷贝自定义实现

FS之目录操作 API

常见目录操作 API

- `access`: 判断文件或目录是否具有操作权限
- `stat`: 获取目录及文件信息
- `mkdir`: 创建目录
- `rmdir`: 删除目录
- `readdir`: 读取目录中内容
- `unlink`: 删除指定文件

创建目录之同步实现

创建目录之异步实现

递归删除目录

模块化历程

前端开发为什么需要模块化？

组件化前端开发

传统开发常见问题

- 命名冲突和污染
- 代码冗余，无效请求多
- 文件间的依赖关系复杂

项目难以维护不方便复用

模块就是小而精且利用维护的代码片段

利用函数、对象、自执行函数实现分块

常见模块化规范

- Commonjs 规范
- AMD 规范
- CMD 规范
- ES modules 规范

模块化规范

- 模块化是前端走向工程化中的重要一环
- 早期 JavaScript 语言层面没有模块化规范
- Commonjs 、AMD、CMD、都是模块化规范
- ES6 中将模块化纳入标准规范
- 当下常用规范是 Commonjs 与 ESM

CommonJS 规范

not just for browsers any more!

CommonJS 规范主要应用于 Nodejs

CommonJS 是语言层面的规范

CommonJS 规范

- 模块引用
- 模块定义
- 模块标识

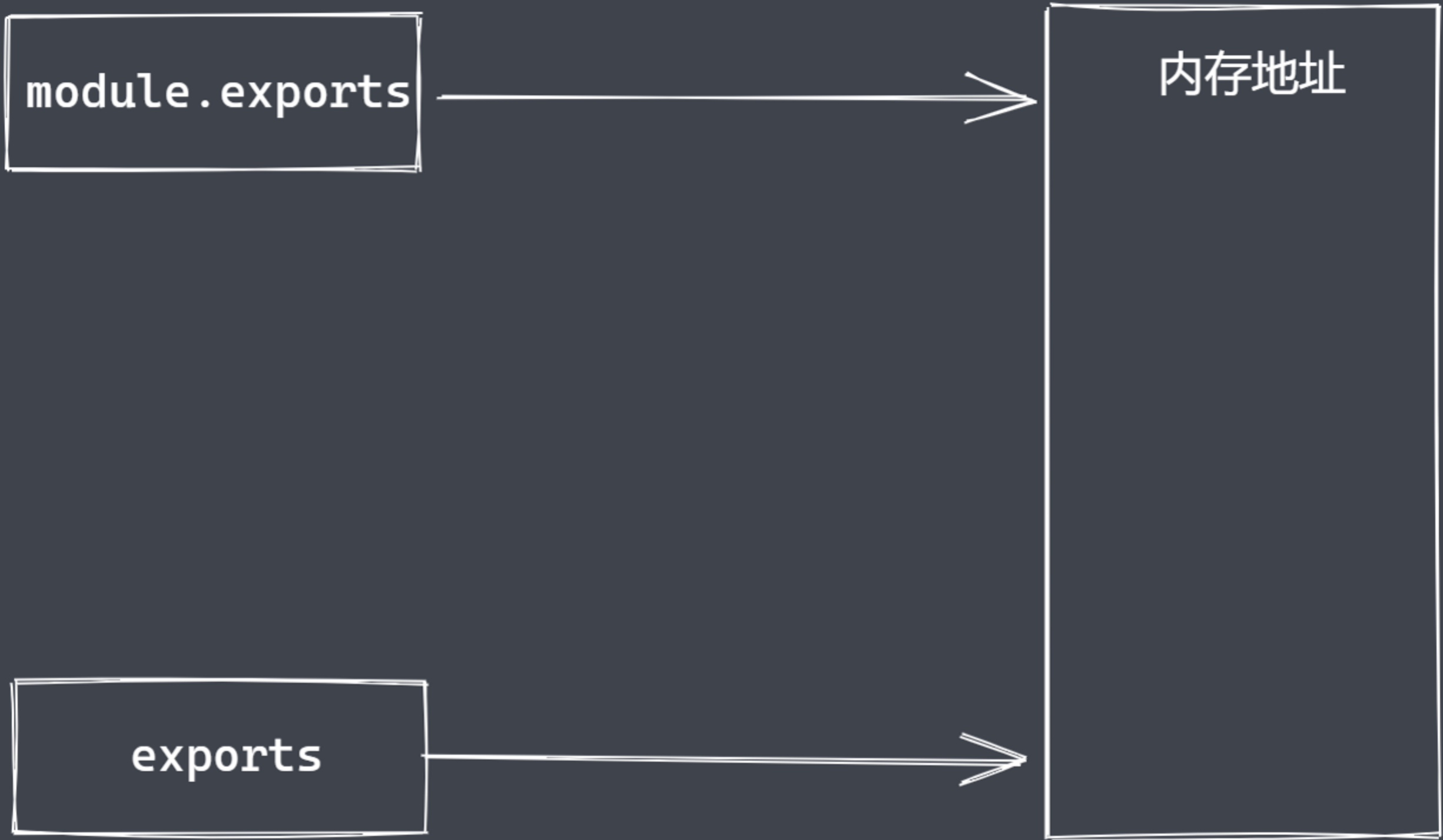
Nodejs与CommonJS

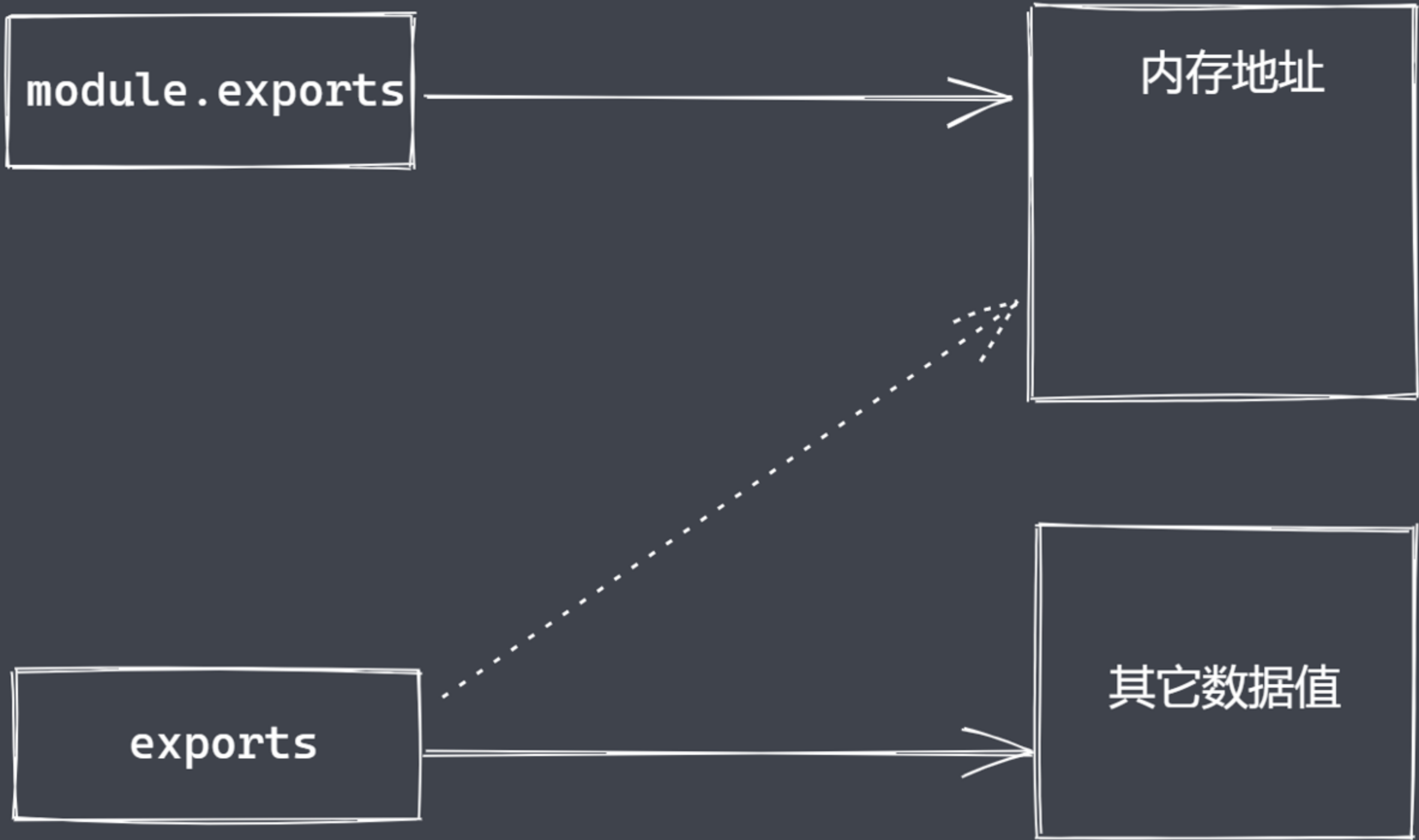
- 任意一个文件就是一模块，具有独立作用域
- 使用 `require` 导入其它模块
- 将模块 ID 传入 `require` 实现目标模块定位

module 属性

- 任意 js 文件就是一个模块，可以直接使用 module 属性
- id: 返回模块标识符，一般是一个绝对路径
- filename: 返回文件模块的绝对路径
- loaded: 返回布尔值，表示模块是否完成加载
- parent: 返回对象存放调用当前模块的模块
- children: 返回数组，存放当前模块调用的其它模块
- exports: 返回当前模块需要暴露的内容
- paths: 返回数组，存放不同目录下的 node_modules 位置

module.exports 与 exports 有何区别？





require 属性

- 基本功能是读入并且执行一个模块文件
- `resolve`: 返回模块文件绝对路径
- `extensions`: 依据不同后缀名执行解析操作
- `main`: 返回主模块对象

CommonJS 规范

- CommonJS 规范起初是为了弥补 JS 语言模块化缺陷
- CommonJS 是语言层面的规范，当前主要用于 Node.js
- CommonJS 规定模块化分为引入、定义、标识符三个部分
- Module 在任意模块中可直接使用包含模块信息
- Require 接收标识符，加载目标模块
- Exports 与 module.exports 都能导出模块数据
- CommonJS 规范定义模块的加载是同步完成

Node.js 与 CommonJS

Node.js 与 CommonJS

- 使用 `module.exports` 与 `require` 实现模块导入与导出
- `module` 属性及其常见信息获取
- `exports` 导出数据及其与 `module.exports` 区别
- CommonJS 规范下的模块同步加载

模块加载源码分析

内置模块之 VM

创建独立运行的沙箱环境

模块加载模拟实现

模拟文件模块加载流程实现

核心逻辑

- 路径分析
- 缓存优化
- 文件定位
- 编译执行

核心模块之 Events

通过 EventEmitter 类实现事件统一管理

events 与 EventEmitter

- node.js 是基于事件驱动的异步操作架构，内置 events 模块
- events 模块提供了 EventEmitter 类
- node.js 中很多内置核心模块继承 EventEmitter

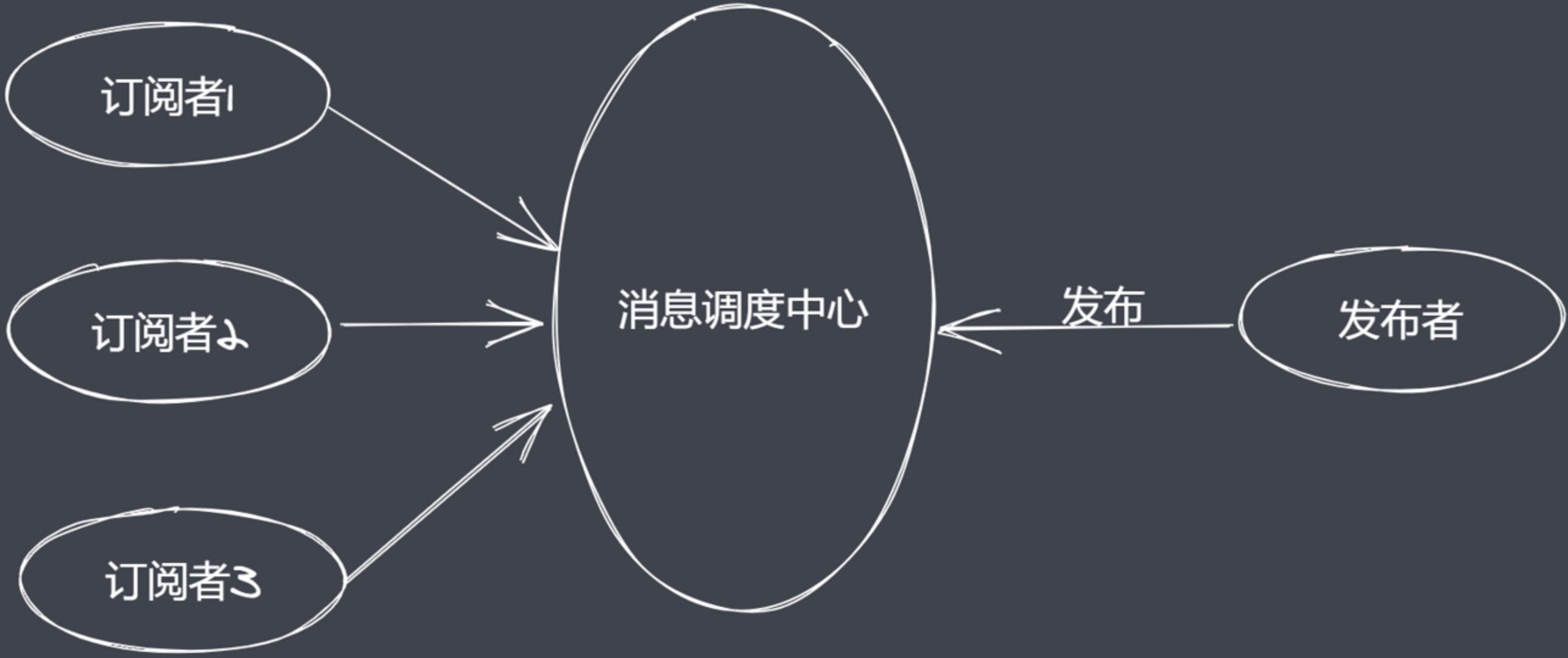
EventEmitter 常见 API

- `on`: 添加当事件被触发时调用的回调函数
- `emit`: 触发事件, 按照注册的序同步调用每个事件监听器
- `once` : 添加当事件在注册之后首次被触发时调用的回调函数
- `off`: 移除特定的监听器

发布订阅模式

定义对象间一对多的依赖关系

发布订阅可以解决什么问题？



发布订阅要素

- 缓存队列，存放订阅者信息
- 具有增加、删除订阅的能力
- 状态改变时通知所有订阅者执行监听

发布订阅中存在调度中心

状态发生改变时，发布订阅无须主动通知

EventEmitter 类源码分析

EventEmitter 模拟

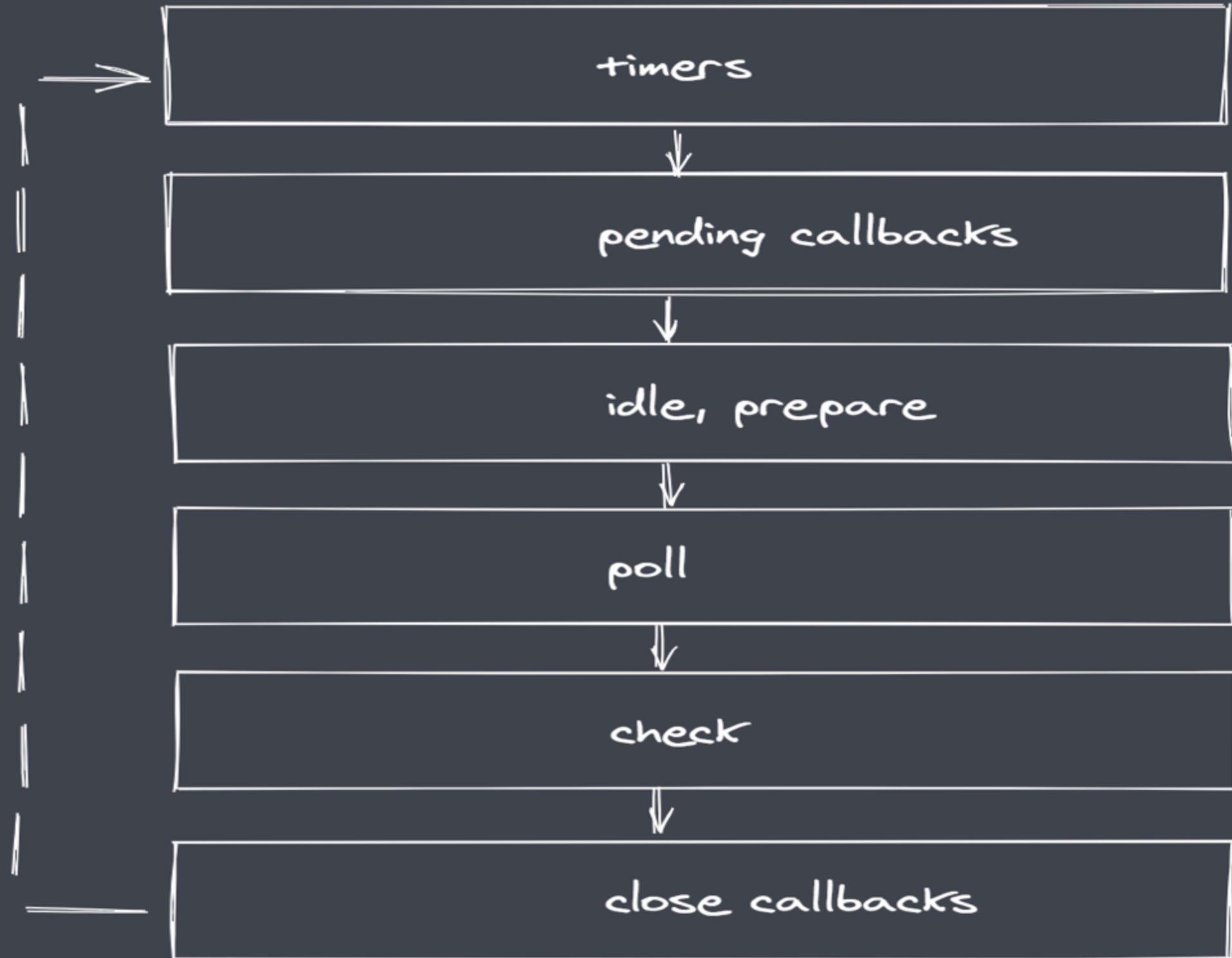
浏览器中的 Event loop

完整事件环执行顺序

- 从上至下执行所有的同步代码
- 执行过程中将遇到的宏任务与微任务添加至相应的队列
- 同步代码执行完毕后，执行满足条件的微任务回调
- 微任务队列执行完毕后执行所有满足需求的宏任务回调
- 循环事件环操作
- 注意：每执行一个宏任务之后就会立刻检查微任务队列

Nodejs 下的事件环

Nodejs 事件循环机制



队列说明

- `timers`: 执行 `setTimeout` 与 `setInterval` 回调
- `pending callbacks`: 执行系统操作的回调, 例如 `tcp udp`
- `idle, prepare`: 只在系统内部进行使用
- `poll`: 执行与 `I/O` 相关的回调
- `check`: 执行 `setImmediate` 中的回调
- `close callbacks`: 执行 `close`事件的回调

Nodejs 完整事件环

- 执行同步代码，将不同的任务添加至相应的队列
- 所有同步代码执行后会去执行满足条件微任务
- 所有微任务代码执行后会执行 timer 队列中满足的宏任务
- timer 中的所有宏任务执行完成后就会依次切换队列
- 注意：在完成队列切换之前会先清空微任务代码

Nodejs 事件环梳理

Nodejs 与浏览器事件环区别

Node 与浏览器事件环不同

- 任务队列数不同
- Nodejs 微任务执行时机不同
- 微任务优先级不同

任务队列数

- 浏览器中只有二个任务队列
- Nodejs 中有6个事件队列

微任务执行时机

- 二者都会在同步代码执行完毕后执行微任务
- 浏览器平台下每当一个宏任务执行完毕后就清空微任务
- Nodejs 平台在事件队列切换时会去清空微任务

微任务优先级

- 浏览器事件环中，微任务存放于事件队列，先进先出
- Nodejs 中 `process.nextTick` 先于 `promise.then`

Nodejs 事件环常见问题

核心模块之 Stream

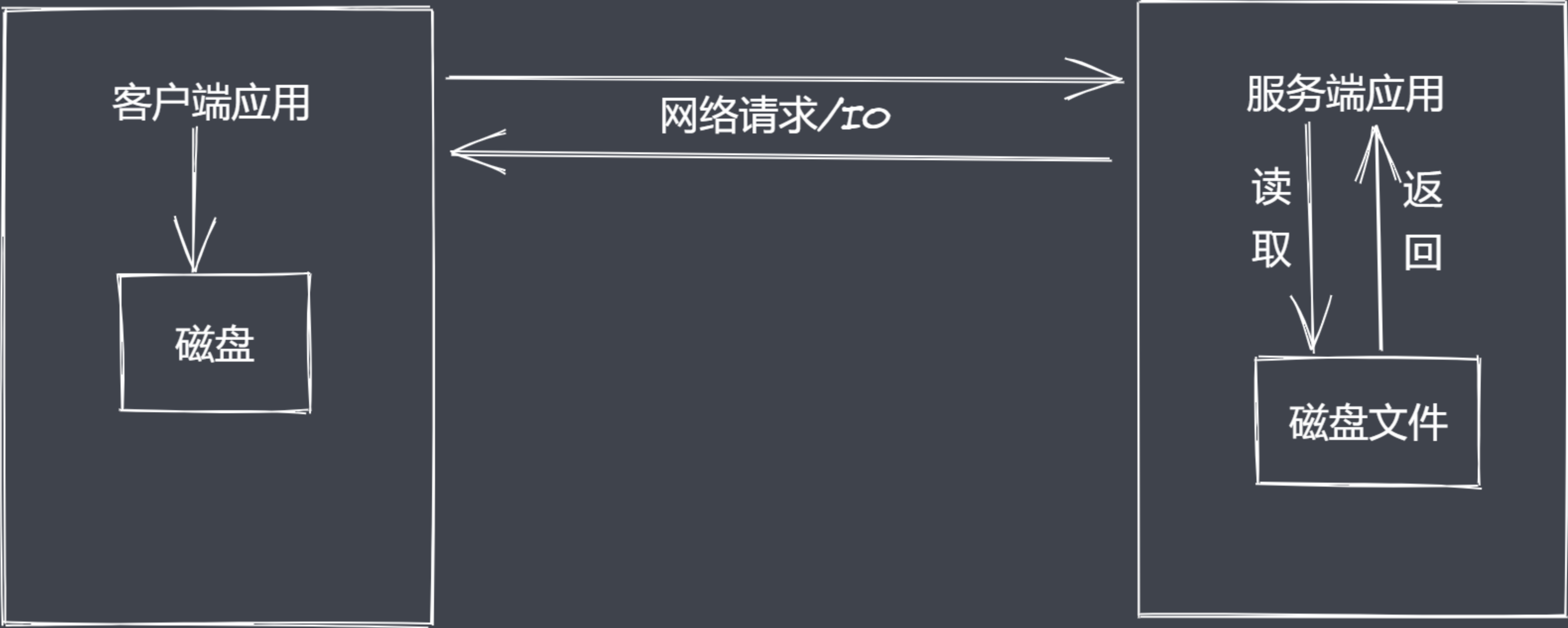
```
ls | grep *.js
```

Node.js 诞生之初就是为了提高 IO 性能

文件操作系统和网络模块实现了流接口

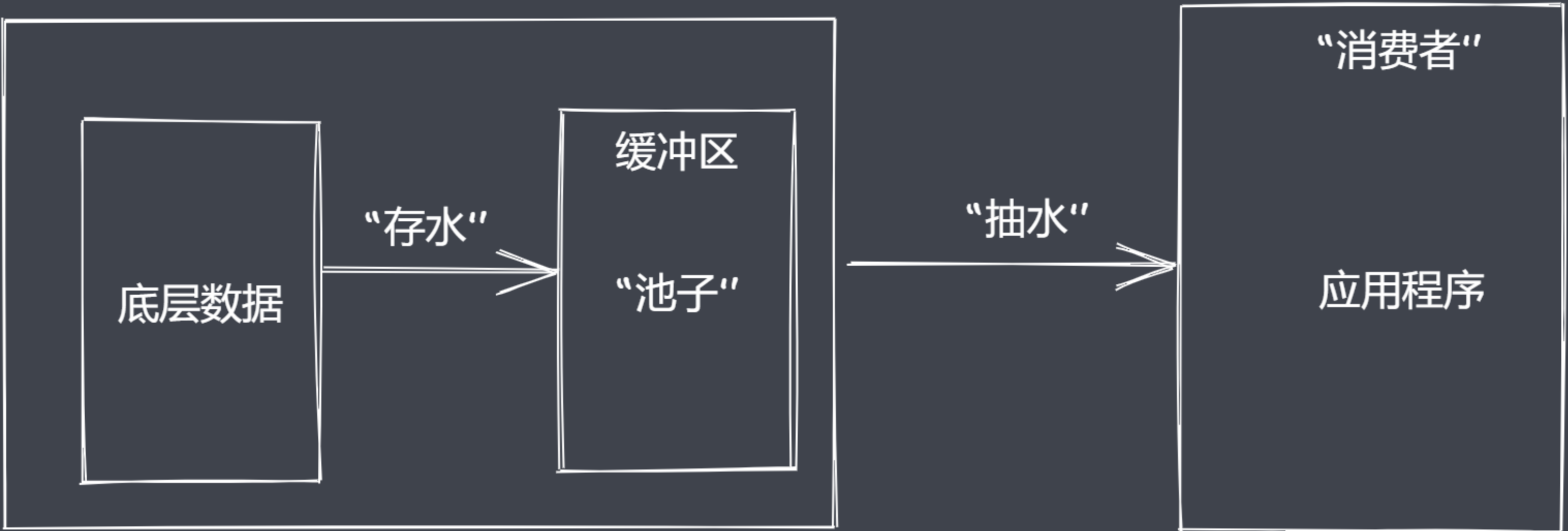
Node.js 中的流就是处理流式数据的抽象接口

应用程序中为什么使用流来处理数据？



常见问题

- 同步读取资源文件，用户需要等待数据读取完成
- 资源文件最终一次性加载至内存，开销较大





流处理数据的优势

- 时间效率：流的分段处理可以同时操作多个数据 chunk

流处理数据的优势

- 时间效率：流的分段处理可以同时操作多个数据 chunk
- 空间效率：同一时间流无须占据大内存空间

流处理数据的优势

- 时间效率：流的分段处理可以同时操作多个数据 chunk
- 空间效率：同一时间流无须占据大内存空间
- 使用方便：流配合管理，扩展程序变得简单

Node.js 内置了 stream ，它实现了流操作对象

Node.js 中流的分类

- Readable: 可读流, 能够实现数据的读取
- Writable: 可写流, 能够实现数据的写操作
- Duplex: 双工流, 既可读又可写
- Transform: 转换流, 可读可写, 还能实现数据转换

Node.js 流特点

- Stream 模块实现了四个具体的抽象
- 所有流都继承自 EventEmitter

可读流

“生产供程序消费数据的流”



```
const fs = require('fs')  
  
const rs = fs.createReadStream('00-note.txt')  
  
rs.pipe(process.stdout)
```


如何自定义可读流？

自定义可读流

- 继承 stream 里的 Readable
- 重写 _read 方法调用 push 产出数据

```
const {Readable} = require('stream')
// 定义数组存放数据, 模拟底层数据
let source = ['lg', 'zce', 'syy']
// 自定义类继承 Readable
class MyReadable extends Readable{
  constructor(source) {
    super()
    this.source = source
  }
  _read() {
    let data = this.source.shift() || null
    this.push(data)
  }
}
// 消费数据
let myReadable = new MyReadable(source)

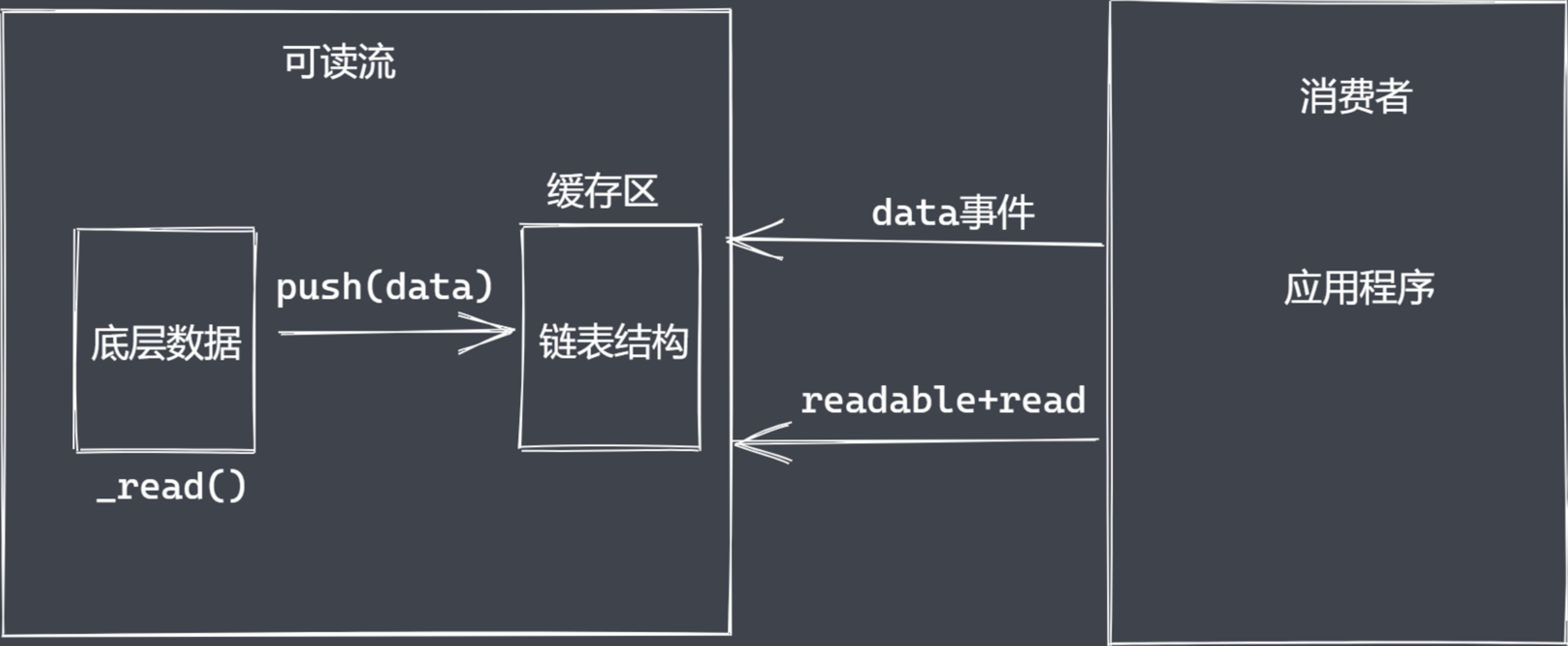
myReadable.on('readable', () => {
  let data = null
  while((data = myReadable.read()) !== null){
    console.log(data.toString())
  }
})
myReadable.on('data', (data) => {
  console.log(data.toString())
})
```


自定义可读流问题

- 底层数据读取完成之后如何处理？
- 消费者如何获取可读流中的数据？

消费数据为什么存在二种方式？

流动模式、暂停模式



消费数据

- readable 事件：当流中存在可读取数据时触发
- data 事件：当流中数据块传给消费者后触发

可读流总结

- 明确数据生产与消费流程
- 利用 API 实现自定义的可读流
- 明确数据消费的事件使用

可写流

“用于消费数据的流”



```
const fs = require('fs')

// 创建一个可读流, 生产数据
let rs = fs.createReadStream('00-note.txt')

// 修改字符编码, 便于后续使用
rs.setEncoding('utf-8')

// 创建一个可写流, 消费数据
let ws = fs.createWriteStream('01-note.txt')

// 监听事件调用方法完成具体的消费
rs.on('data', (chunk) => {
  // 执行数据写入
  ws.write(chunk)
})
```

自定义可写流

- 继承 `stream` 模块的 `Writable`
- 重写 `_write` 方法，调用 `write` 执行写入



```
const {Writable} = require('stream')

class MyWriteable extends Writable{
  constructor(){
    super()
  }
  _write(chunk, en, done) {
    process.stdout.write(chunk.toString() + '---')
    // 写完之后再调用 done
    process.nextTick(done)
  }
}
// 创建可写流用于消费数据
let myWriteable = new MyWriteable()

// 执行数据写入
myWriteable.write('拉勾教育', 'utf-8', () => {
  console.log('写入成功了')
})
```

可写流事件

- pipe事件：可读流调用 `pipe()` 方法时触发
- unpipe事件：可读流调用 `unpipe()` 方法时触发
- drain 事件：`write`返回`false`，数据可执行写入时触发

Duplex && Transform

Node.js 中 stream 是流操作的抽象接口集合

可读、可写、双工、转换是单一抽象具体实现

流操作的核心功能就是处理数据

Node.js 诞生的初衷就是解决密集型 IO 事务

Node.js 中处理数据模块继承了流和 EventEmitter

stream、四种类型流、实现流操作的模块

Duplex 是双工流，既能生产又能消费

自定义双工流

- 继承 Duplex 类
- 重写 _read 方法，调用 push 生产数据
- 重写 _write 方法，调用 write 消费数据


```
let {Readable, Writable, Duplex} = require('stream')

// 模拟底层数据源
let source = ['a', 'b', 'c']

// 自定义双工流
class MyDuplex extends Duplex{
  constructor(options) {
    super(source, options)
    this.source = source
  }
  _read() {
    let data = this.source.shift() || null
    this.push(data)
  }
  _write(chunk, enc, next) {
    if (Buffer.isBuffer(chunk)) {
      chunk = chunk.toString()
    }
    process.stdout.write(chunk + '-----')
    process.nextTick(next)
  }
}

// 实例化
let myDuplex = new MyDuplex(source)

myDuplex.on('data', (chunk) => {
  console.log(chunk.toString())
})

myDuplex.write('测试数据', 'utf-8', () => {
  console.log('双工流测试可写操作')
})
```

Transform 也是一个双工流

自定义转换流

- 继承 Transform 类
- 重写 _transform 方法，调用 push 和 callback
- 重写 _flush 方法，处理剩余数据



```
const {Transform} = require('stream')

class MyTransform extends Transform{
  constructor(options) {
    super()
  }
  _transform(chunk, encoding, callback) {
    this.push(chunk.toString().toUpperCase())
    callback(null)
  }
}

let a = new MyTransform()

a.write('a')
a.write('b')
a.end('c')

a.pipe(process.stdout)
```

Node.js 中的四种流

- Readable 可读流
- Writable 可写流
- Duplex 双工流
- Transform 转换流

文件可读流

继承 Readable 与 EventEmitter

文件可写流

write 执行流程

drain与写入速度

背压机制

Node.js 的 stream 已实现了背压机制

数据读写时可能存在的问题

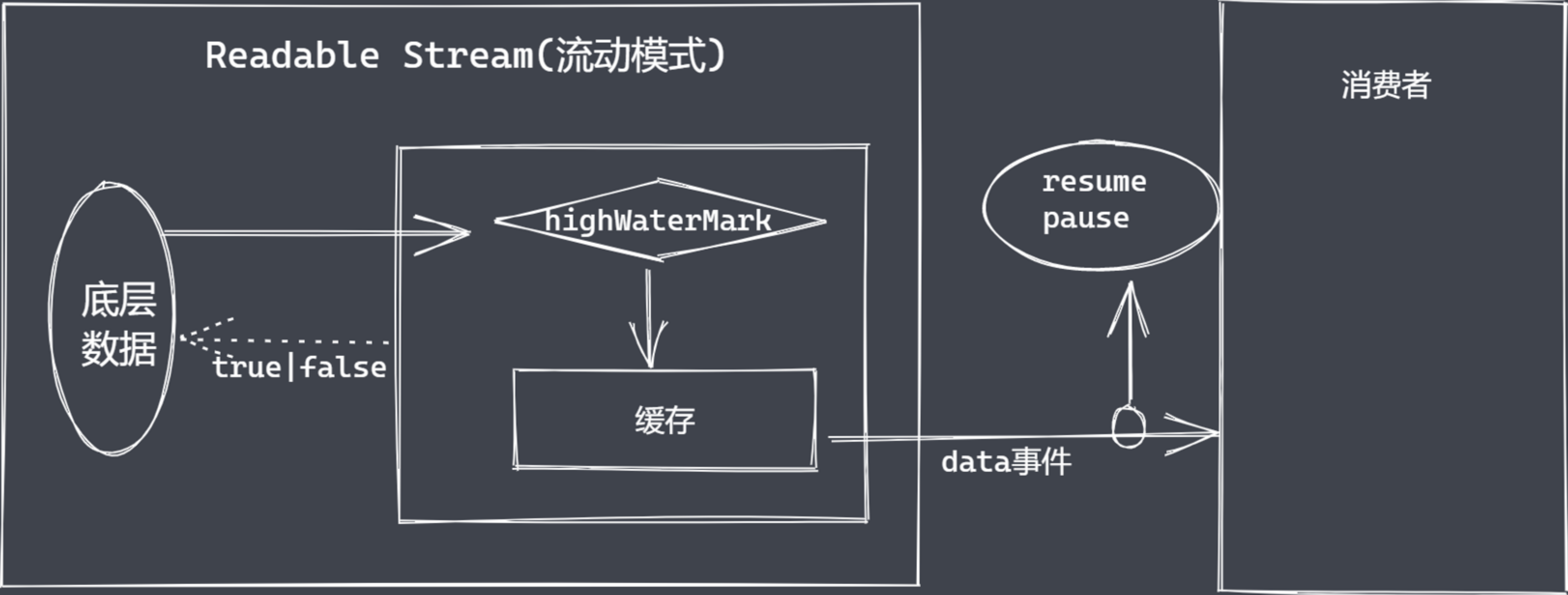


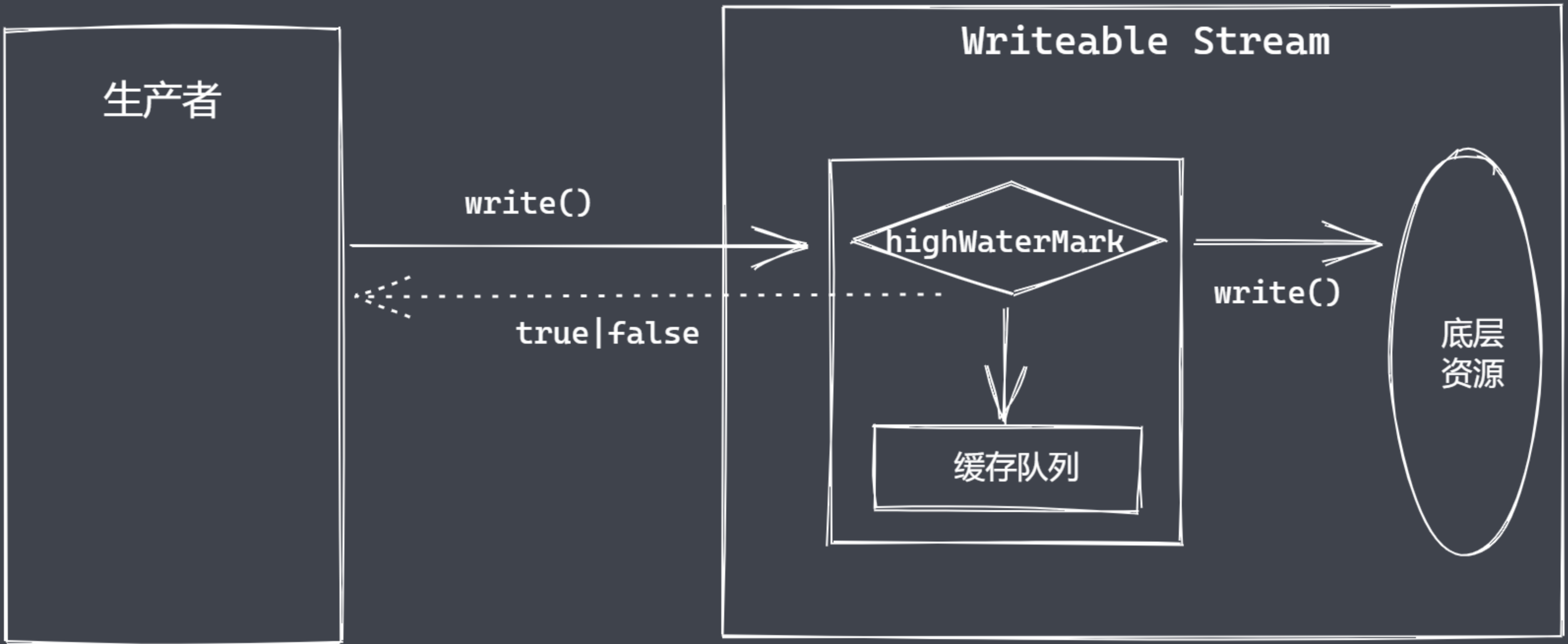
```
const fs = require('fs')

let rs = fs.createReadStream('00-note.txt')
let ws = fs.createWriteStream('00-note1.txt')

rs.on('data', (chunk) => {
  ws.write(chunk)
})
```

内存溢出、GC频繁调用、其它进程变慢





模拟文件可读流

链表结构

为什么不采用数组存储数据？

数组缺点

- 数组存储数据的长度具有上限
- 数组存在塌陷问题

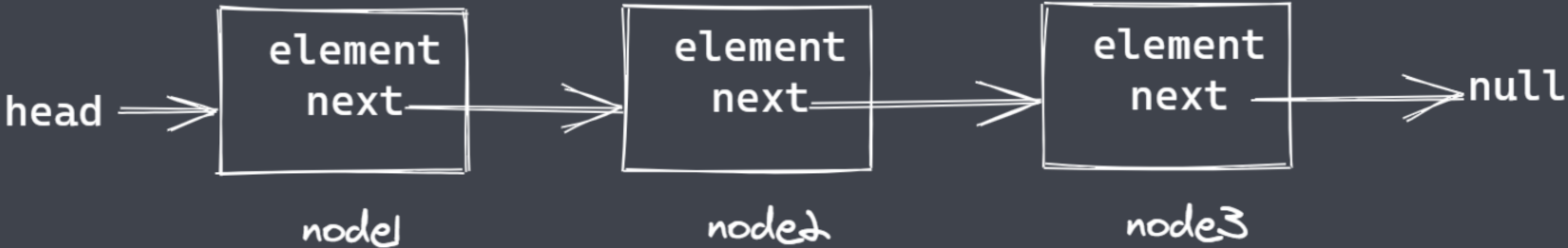
链表是一系列节点的集合

每个节点都具有指向下一个节点的属性

链表分类

- 双向链表
- 单向链表
- 循环链表

LinkedList(链表结构)



单向链表

单向链表实现队列

文件可写流实现

pipe 方法实现

网络通信基本原理

通信必要条件

- 主机之间需要有传输介质
- 主机上必须有网卡设备
- 主机之间需要协商网络速率

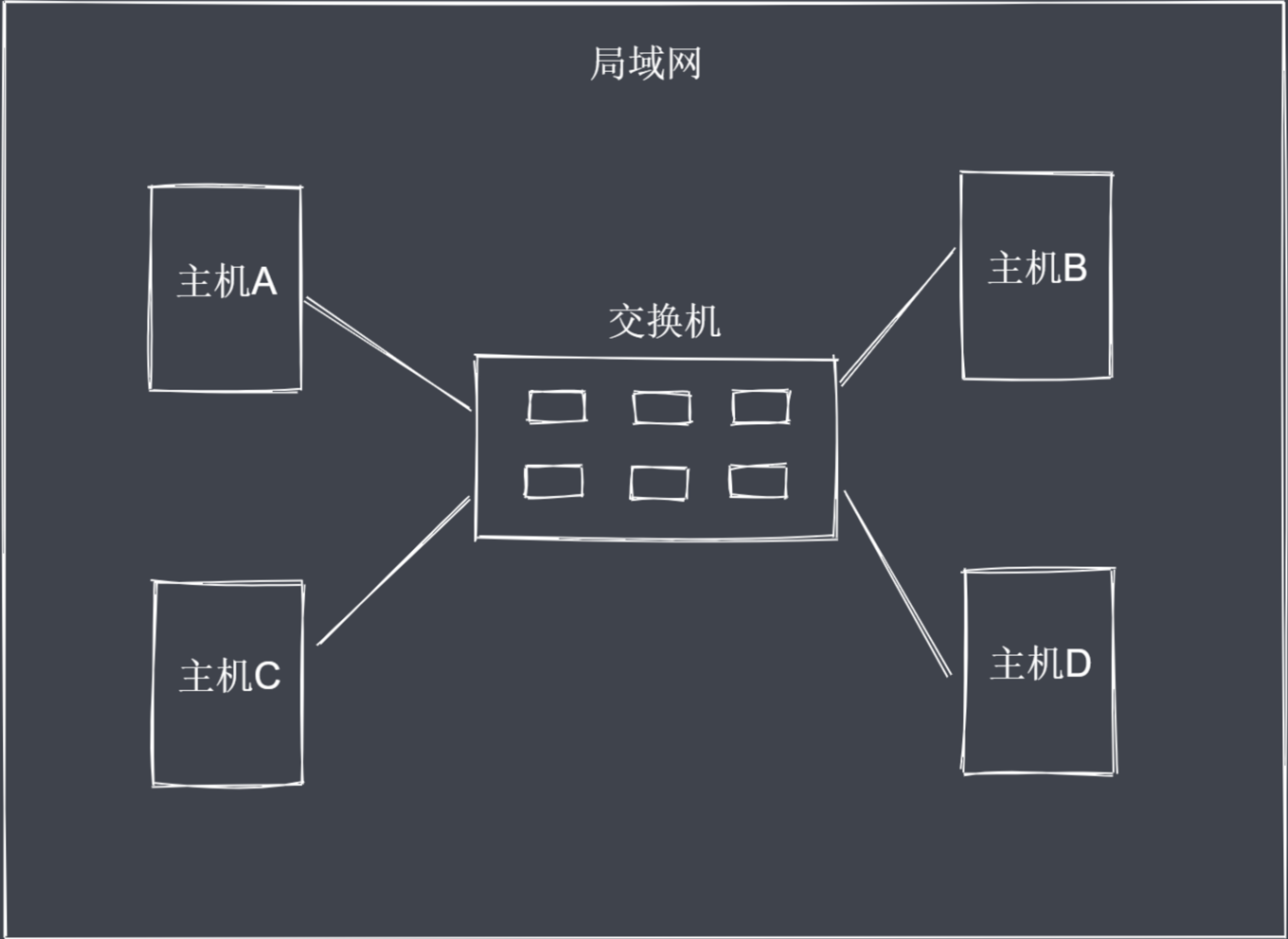


网络通讯方式

常见通讯方式

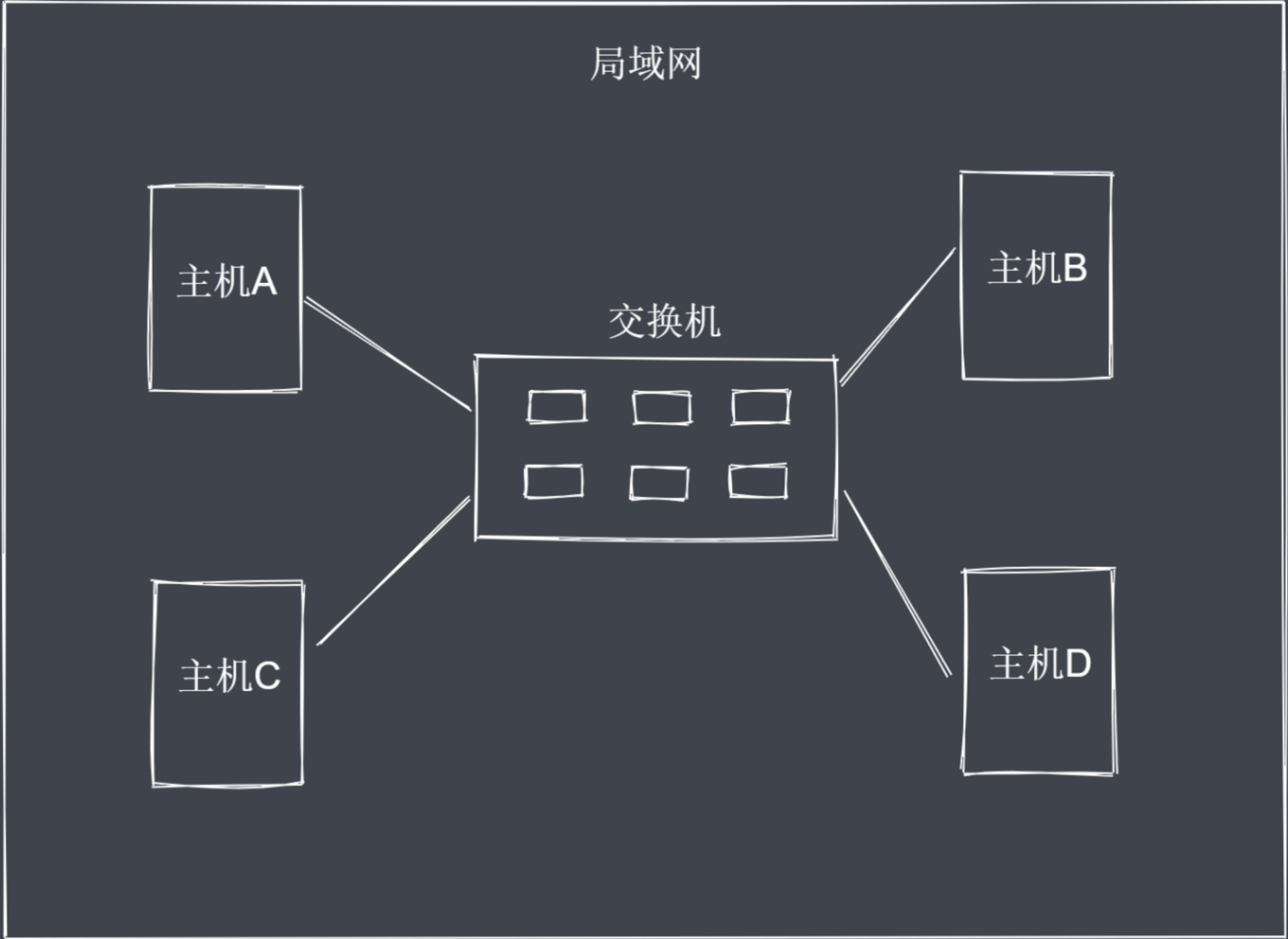
- 交换机通讯
- 路由器通讯

如何建立多台主机互连？



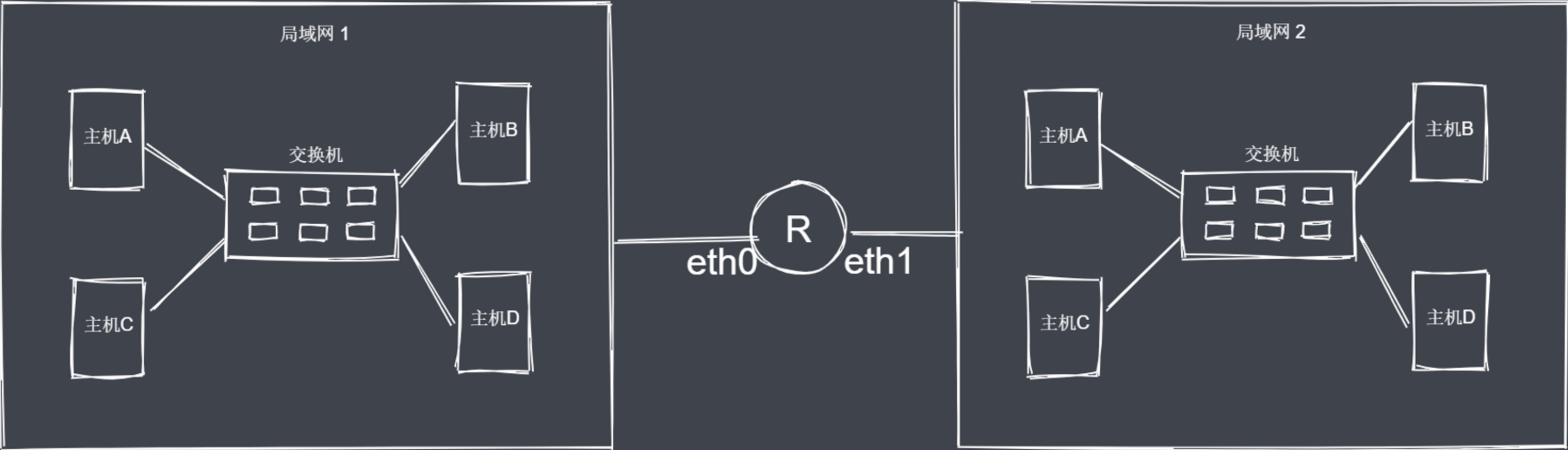
如何定位局域网中的其它主机？

通过 Mac 地址来唯一标识一台主机

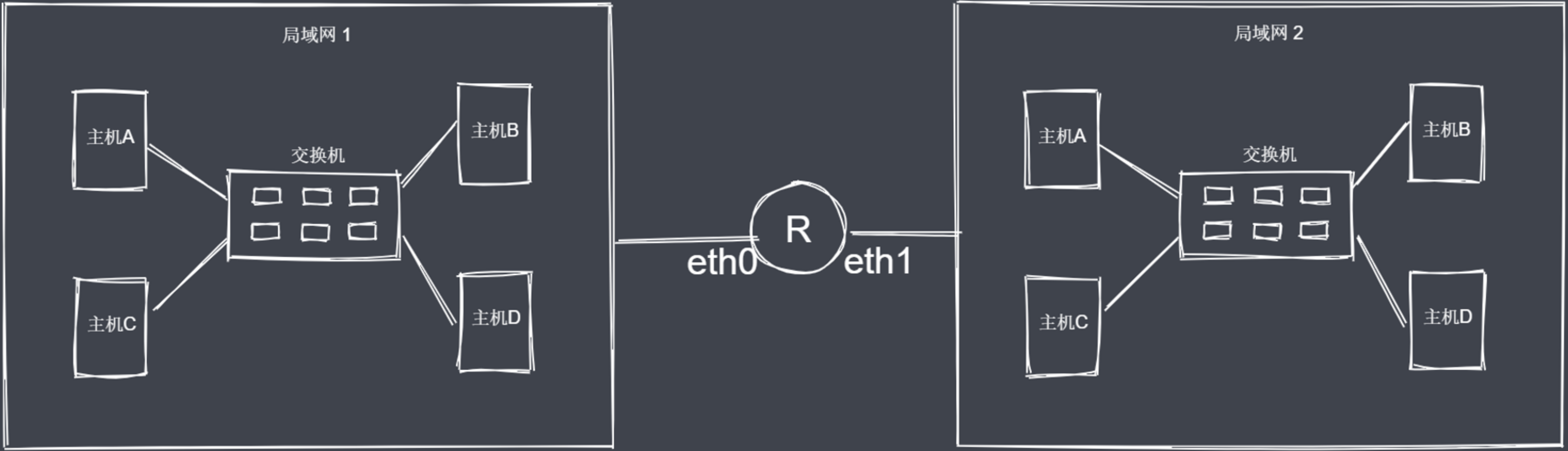


交换机的接口数量有上限

局域网存在大量主机会造成广播风暴



明确目标主机 IP 地址



网络层次模型

OSI 七层模型

- 应用层：用户与网络的接口
- 表示层：数据加密、转换、压缩
- 会话层：控制网络连接建立与终止
- 传输层：控制数据传输可靠性
- 网络层：确定目标网络
- 数据链路层：确定目标主机
- 物理层：各种物理设备和标准

数据从 A 至 B ，先封装再解封

数据封装与解封装

应用层

传输层

网络层

数据链路层

物理层

data

应用层

传输层

网络层

数据链路层

物理层

目标端口

源端口

data





应用层

传输层

网络层

数据链路层

物理层

10001010101001010100010110

应用层

传输层

网络层

数据链路层

物理层

10001010101001010100010110







data

应用层

传输层

网络层

数据链路层

物理层

TCP 三次握手与四次挥手

TCP 协议

- TCP 属于传输层协议
- TCP 是面向连接的协议
- TCP 用于处理实时通信

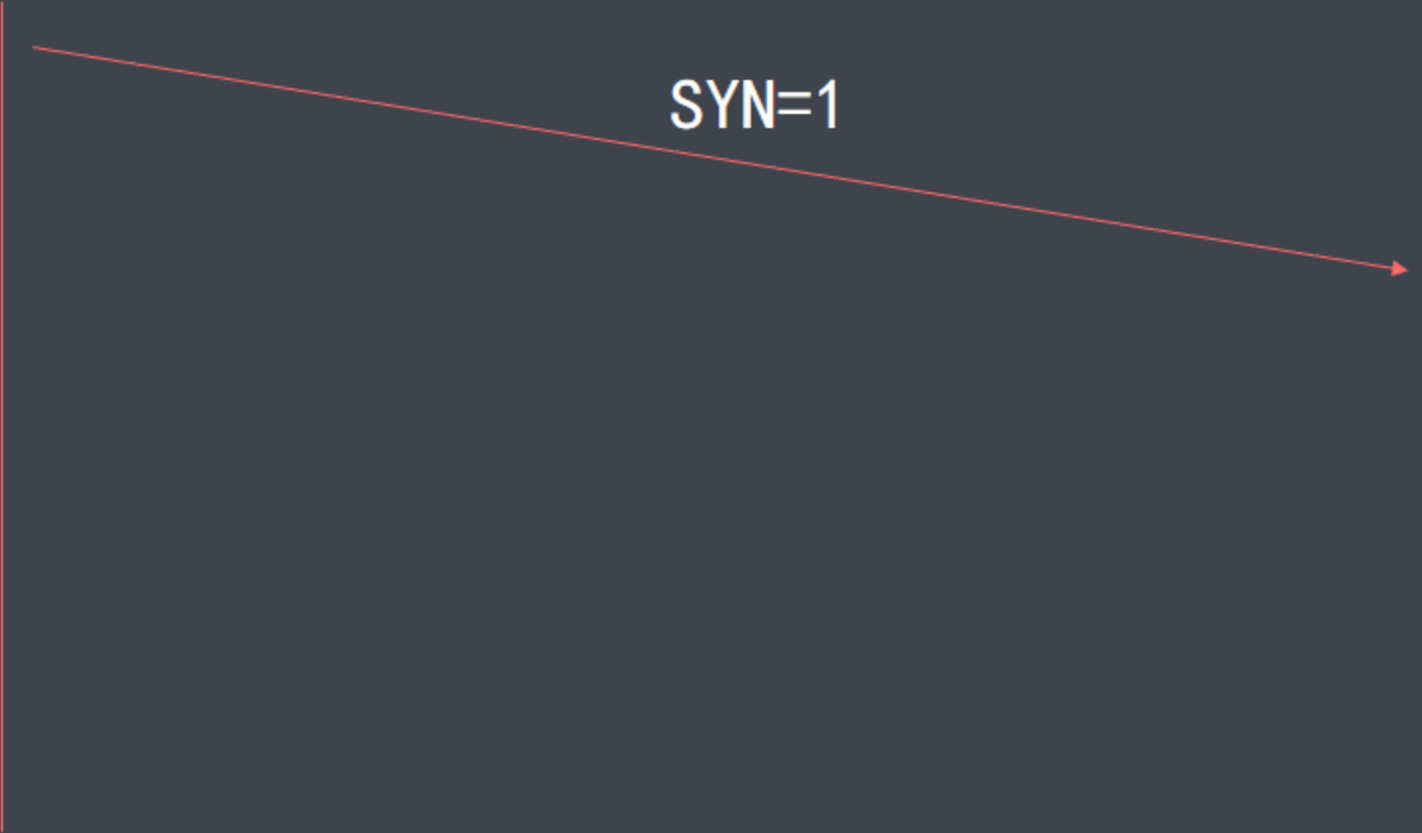


常见控制字段

- SYN = 1 表示请求建立连接
- FIN = 1 表示请求断开连接
- ACK = 1 表示数据信息确认

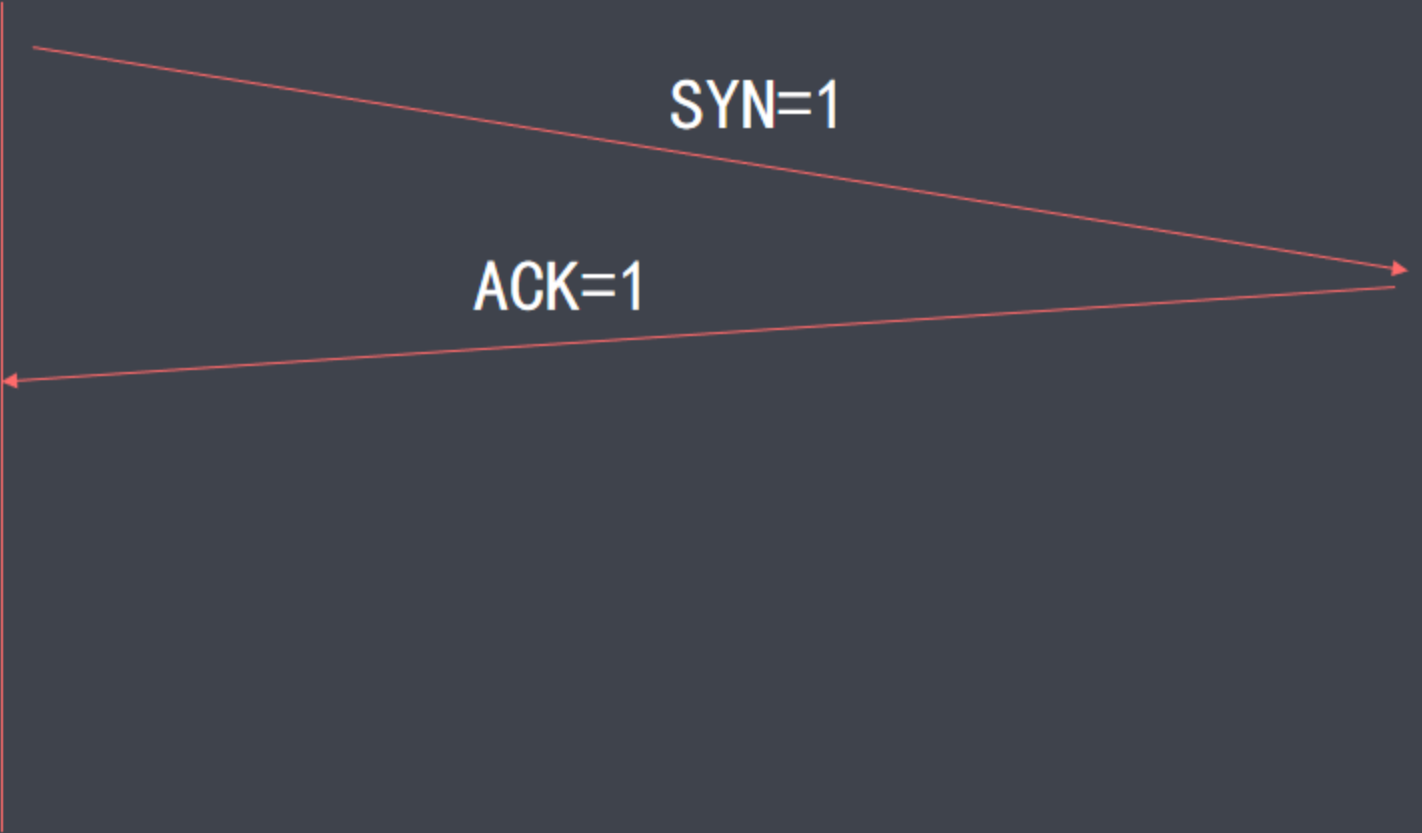
客户端

服务端



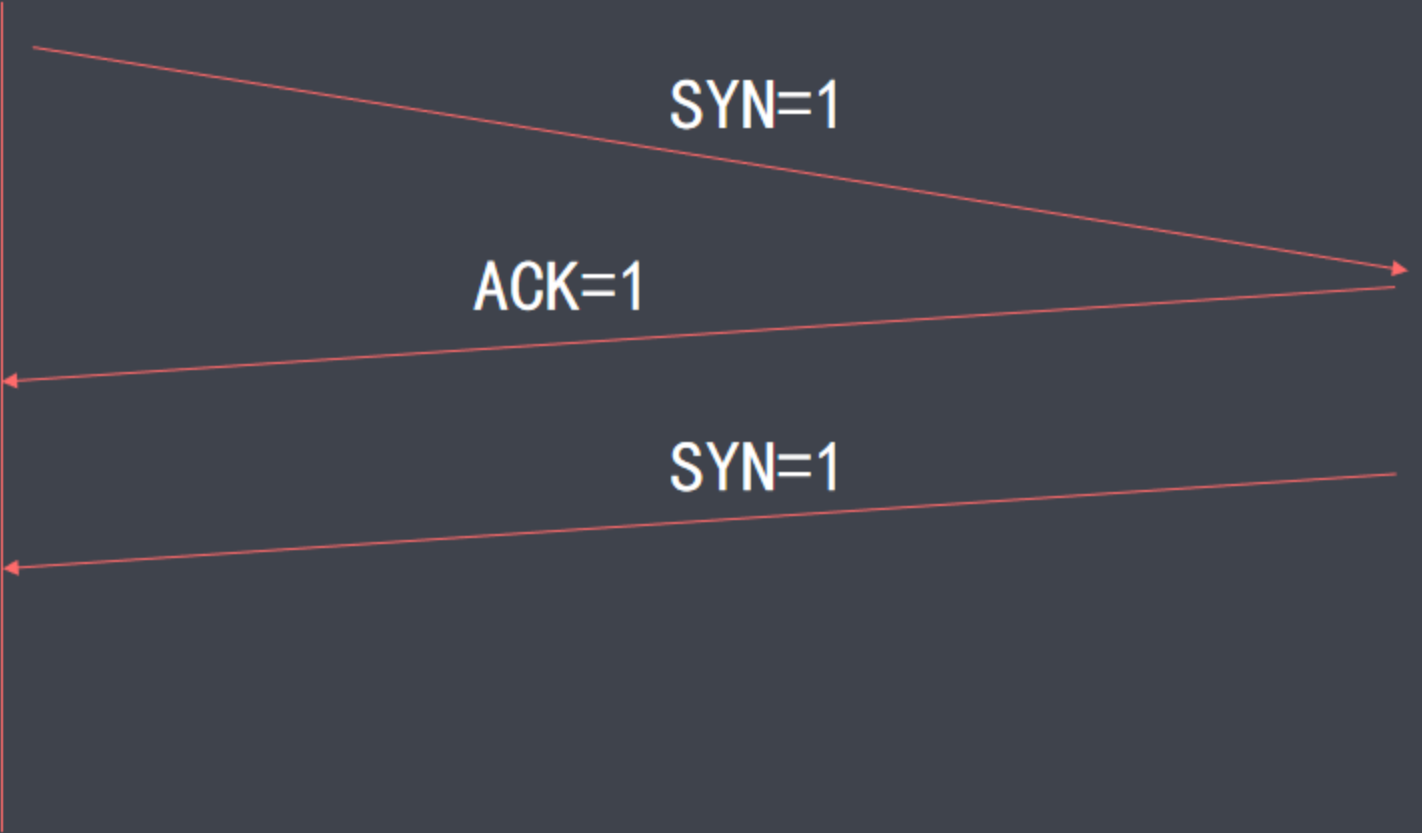
客户端

服务端



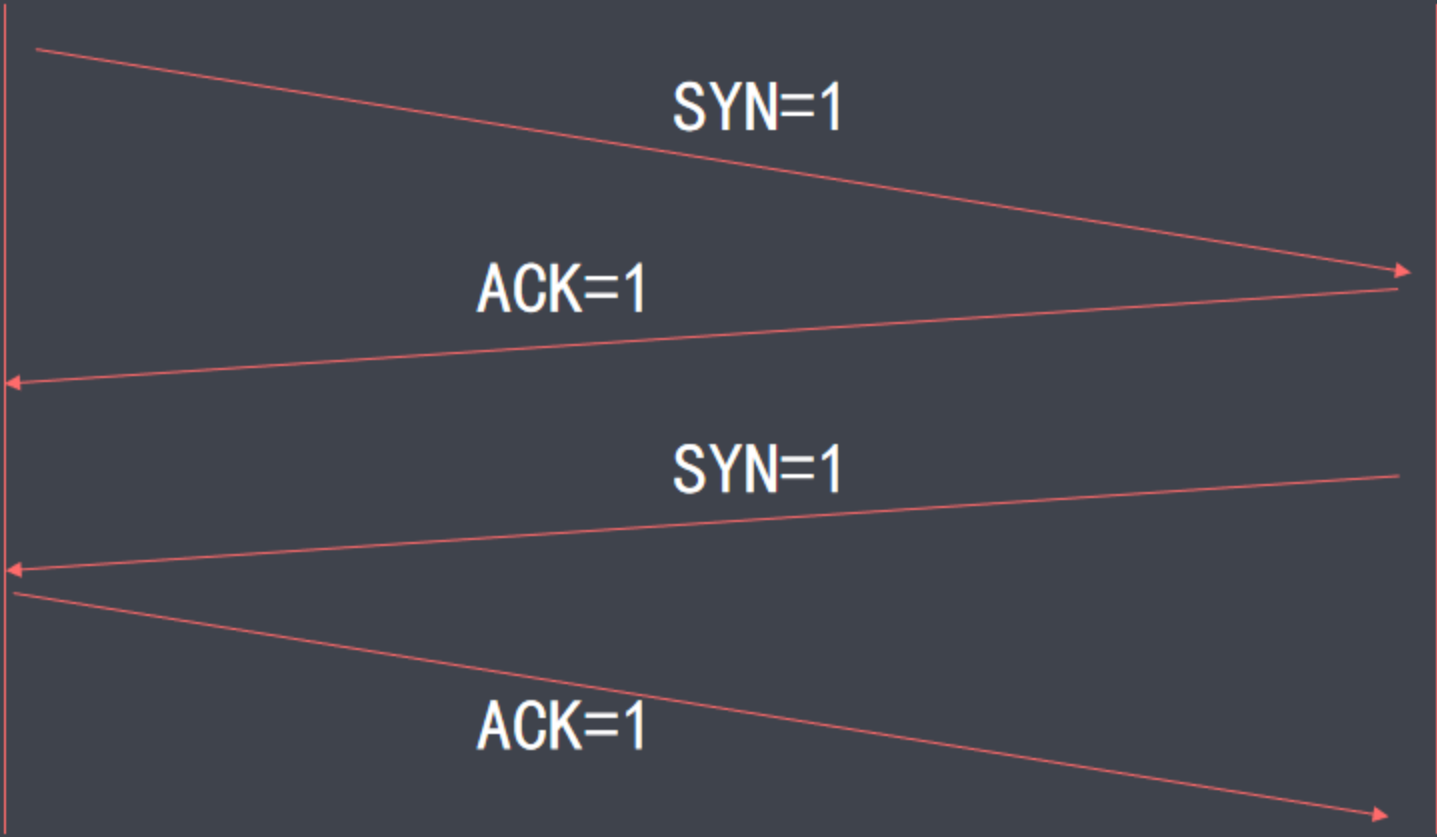
客户端

服务端



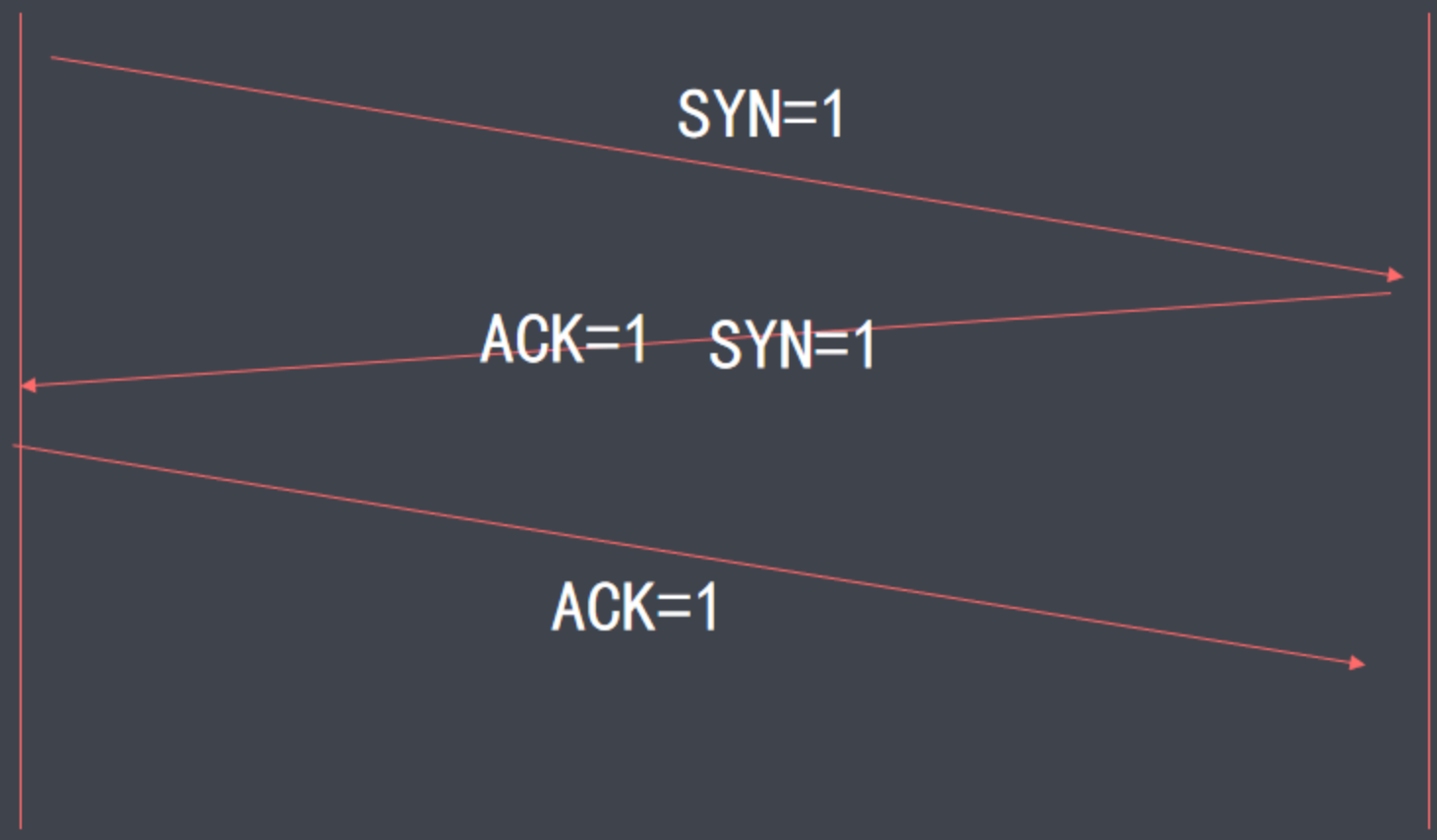
客户端

服务端



客户端

服务端



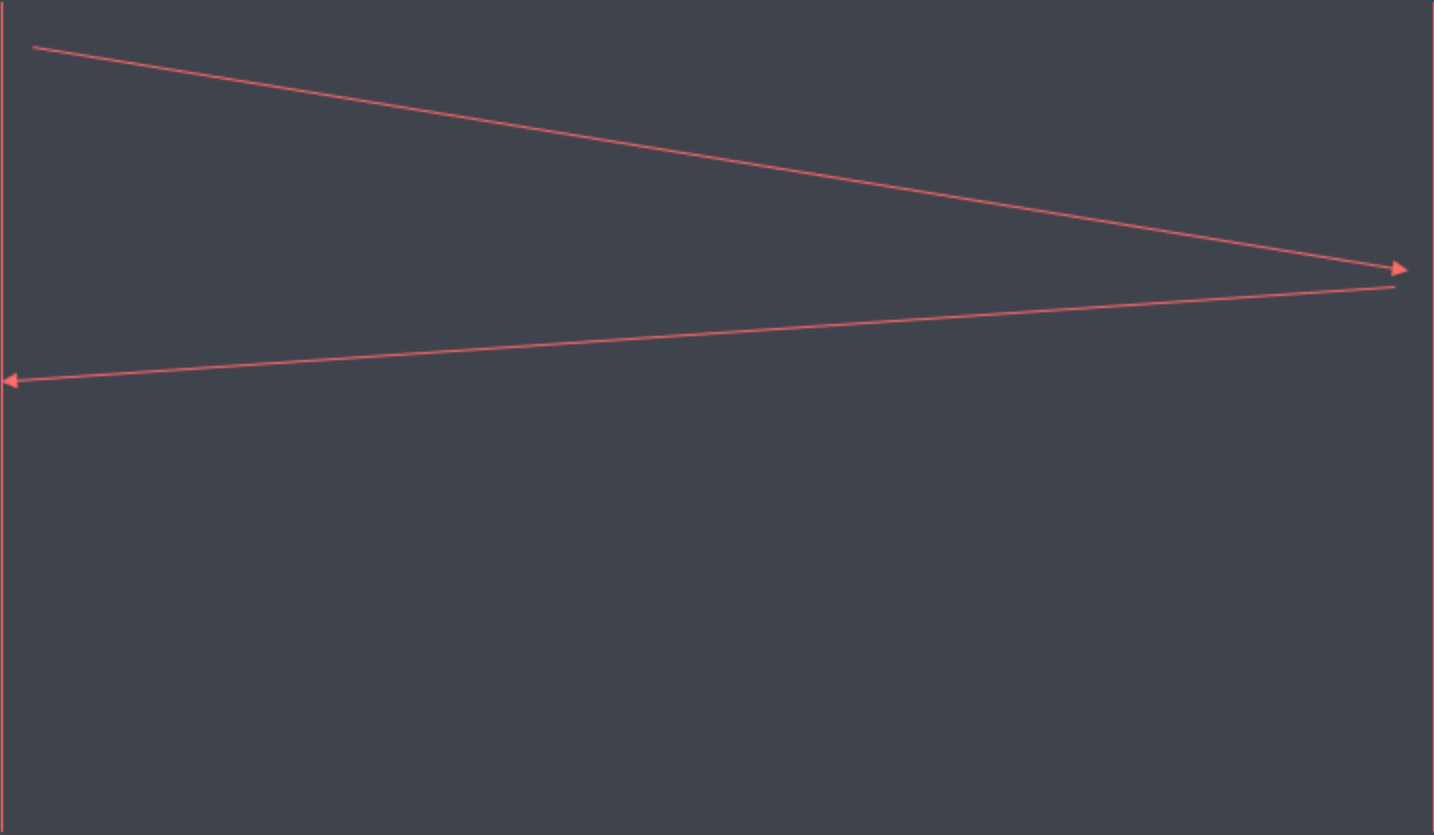
客户端

服务端



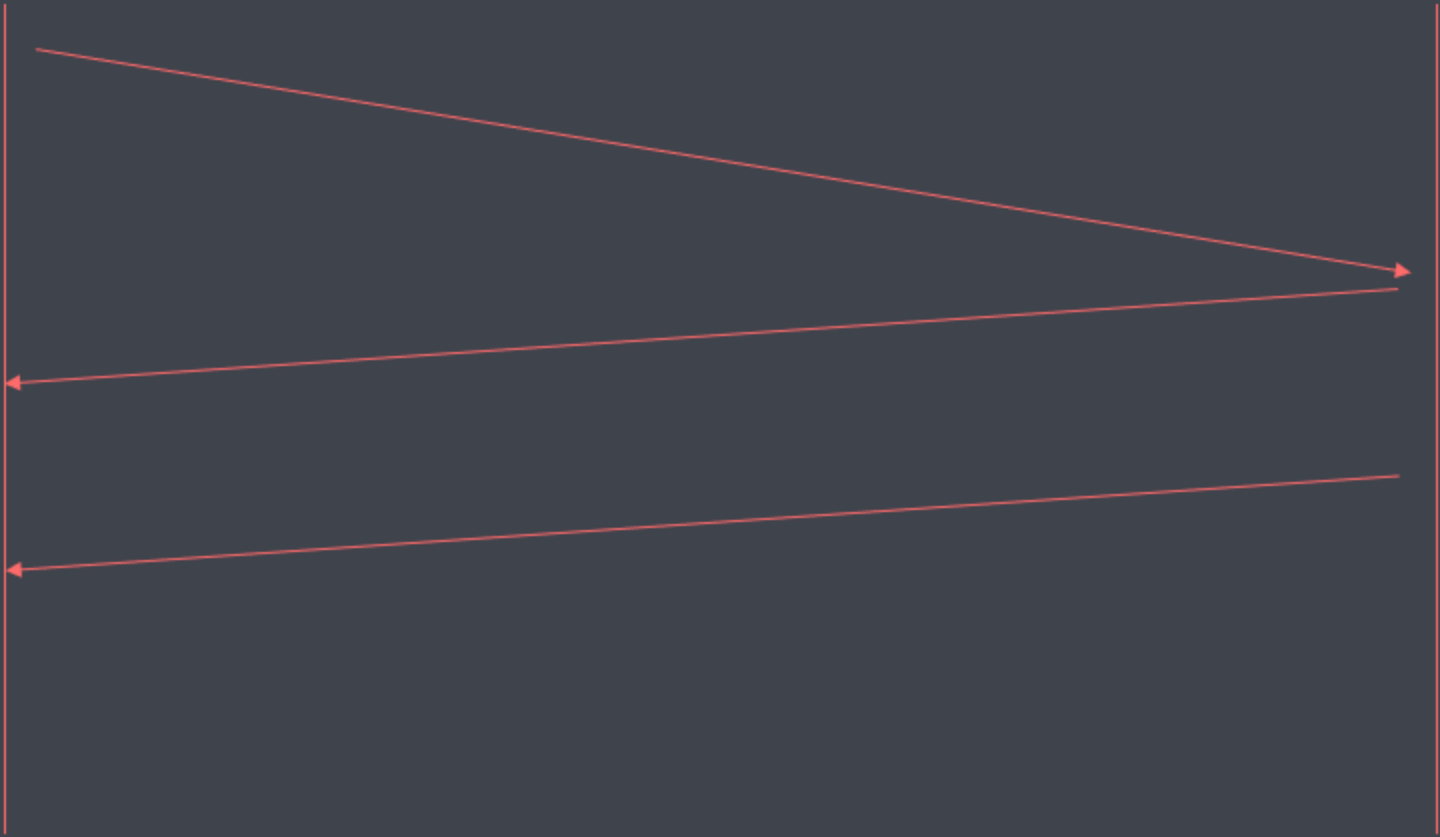
客户端

服务端



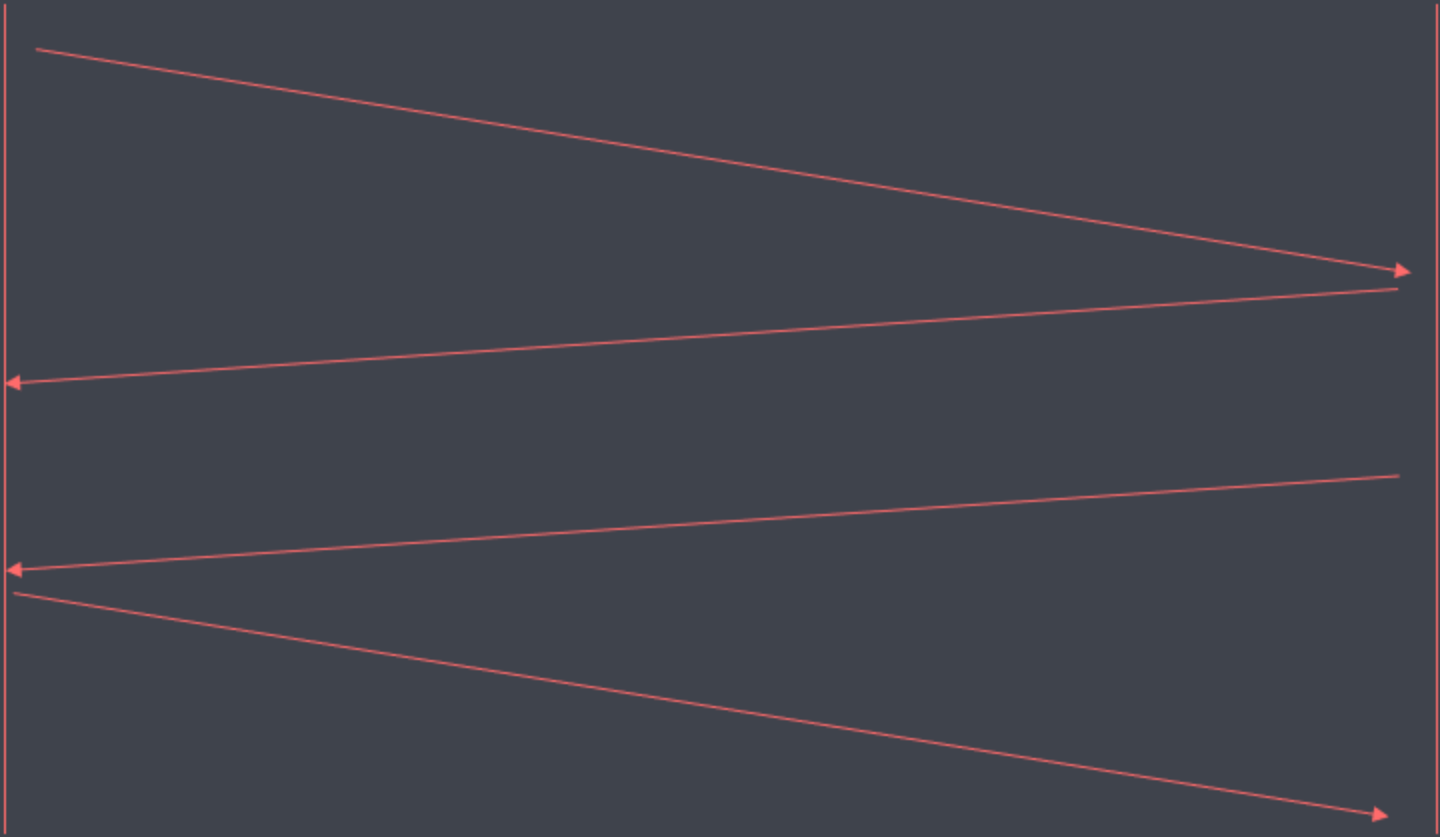
客户端

服务端



客户端

服务端



一个服务端会服务于多个客户端

TCP 协议

- TCP 处于传输层，基于端口，面向连接
- 主机之间要想通信需要先建立双向数据通道
- TCP 的握手和挥手本质上都是四次

创建 TCP 通信

Net 模块实现了底层通信接口

通信过程

- 创建服务端：接收和回写客户端数据
- 创建客户端：发送和接收服务端数据
- 数据传输：内置服务事件和方法读写数据

通信事件

- listening 事件：调用 `server.listen` 方法之后触发
- connection 事件：新的连接建立时触发
- close 事件：当 `server` 关闭时触发
- error 事件：当错误出现的时候触发

通信事件&方法

- data事件：当接收到数据的时候触发该事件
- write方法：在 socket 上发送数据，默认是 UTF8 编码
- end操作：当socket 的一端发送 FIN 包时触发，结束可读端

TCP 数据粘包

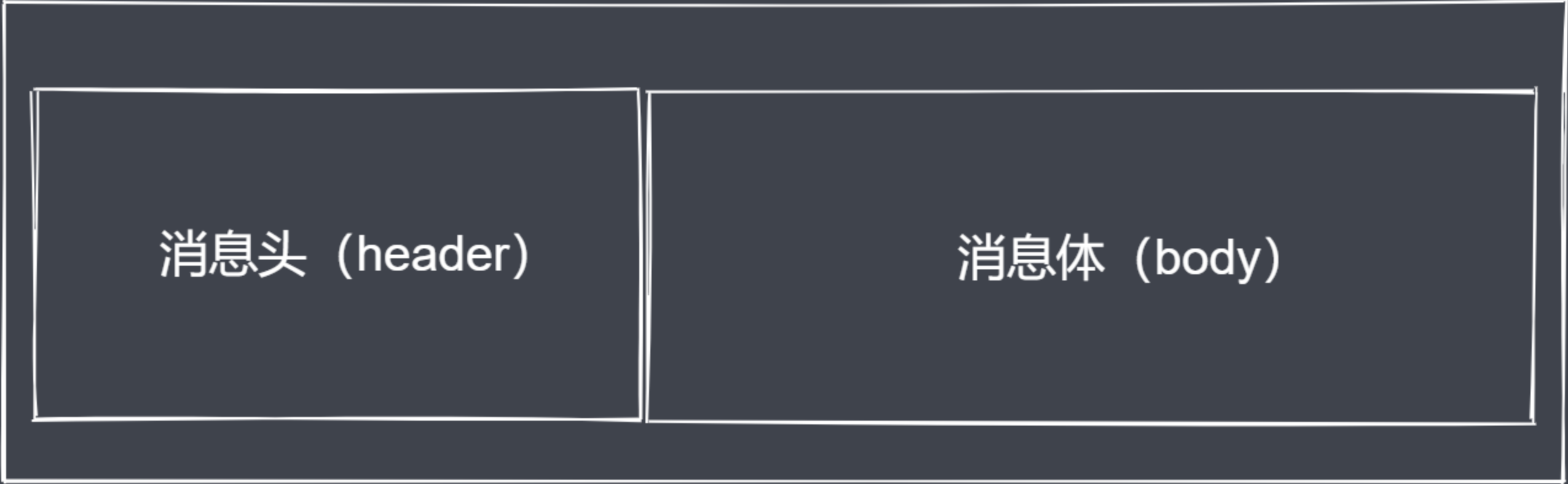
通信包含数据发送端和接收端

发送端累积数据统一发送

接收端缓冲数据之后再消费

TCP 拥塞机制决定发送时机

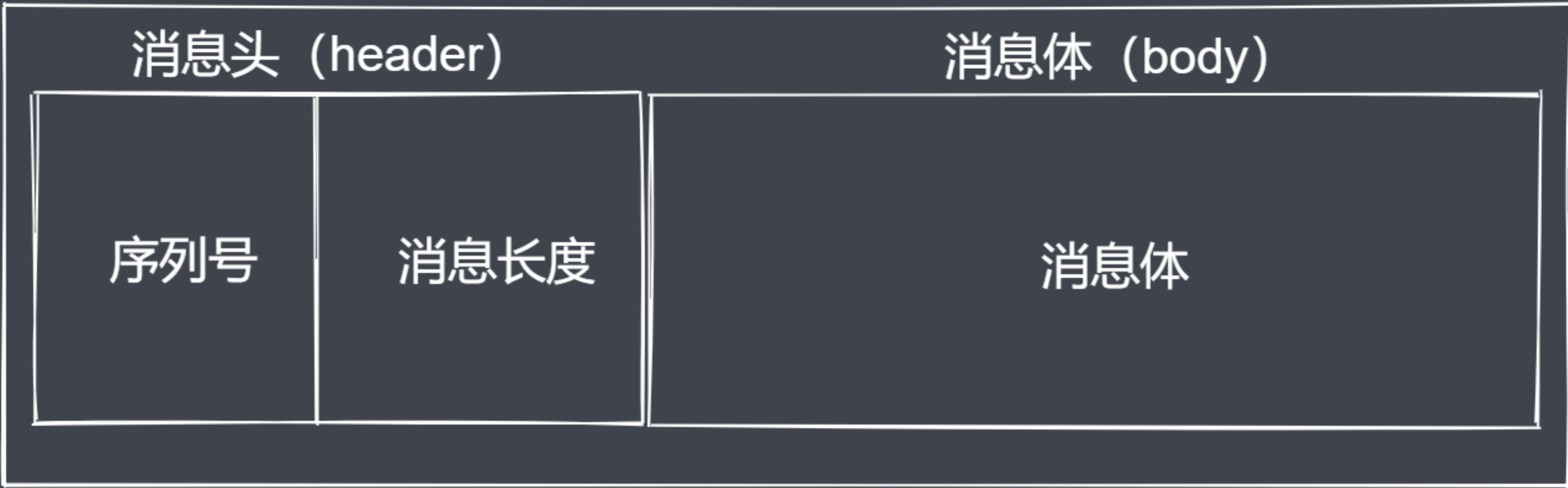
数据的封包与拆包



消息头 (header)

The diagram illustrates the structure of a message. It consists of a large outer rectangle. Inside this rectangle, there is a smaller rectangle divided into two equal-width sections. The left section is labeled '消息头 (header)' and the right section is labeled '消息体 (body)'.

消息体 (body)



数据传输过程

- 进行数据编码，获取二进制数据包
- 按规则拆解数据，获取指定长度的数据

Buffer 数据读写

- `writeInt16BE`: 将 `value` 从指定位置写入
- `readInt16BE`: 从指定位置开始读取数据

Http 协议

获取 Http 请求信息

设置 Http 响应

客户端代理

代理客户端解决跨域

Http 静态服务

静态服务工具